Polytech Montpellier
Université de Montpellier – Année universitaire 2026–2027

Document d'Architecture Technique

Application de gestion de festival

Festival de jeux de société

Client : association « Festival de jeux de société »

Équipe projet (groupe 11)

Kim Etienne – Ines Mazgar – Mathieu Marty

Version : initiale (cible), `dat-v1.3`
Date : 24/10/2026
Dépôt Git : `https://gitlab.polytech.umontpellier.fr/mathieu.marty/awi_kim_mathieu_ines`
Statut : document vivant (journal des évolutions en annexe J (p. 32))

Table des matières

1 Contexte et objectifs **2**

2 Contraintes et exigences non fonctionnelles **2**

3 Vue d'ensemble **3**
3.1 Diagramme de contexte (C4 niveau 1) 4
3.2 Diagramme des briques logicielles (C4 niveau 2) 4

4 Architecture applicative **5**
4.1 Front-end (Angular) 5
4.2 Back-end 6
4.3 Conventions de l'API REST 6

5 Modèle de données **7**

6 Sécurité **7**

7 Architecture de déploiement **9**

8 Build et déploiement (CI/CD) **11**

9 Décisions d'architecture (ADR) **12**

10 Limites et évolutions **12**

Annexes **13**
Liste des figures et des tableaux 13
A Fiches ADR 13
B Glossaire 21
C Matrice des droits 22
D Les niveaux du modèle C4 22
E Besoins, règles de gestion et cas d'utilisation 23
F Comptes rendus des entrevues 25
G Liste complète des endpoints 26
H Rôle des tables du modèle de données 28
I Script SQL initial 29
J Journal des évolutions 32

1 Contexte et objectifs

Finalité. L'association « Festival de jeux de société » loue des tables et des espaces à des exposants (éditeurs, prestataires, l'association elle-même). L'application remplace les suivis manuels : outil de CRM [1] (contacts avec les exposants, festival par festival) et outil de réservation, de placement sur le plan et de facturation, pour plusieurs festivals.

Utilisateurs et rôles. Rôles hiérarchiques, chacun incluant les droits des rôles inférieurs (ADR-001 [2]) : *administrateur* (5), *commercial* (4), *gestionnaire de plan* (3), *gestionnaire de jeux* (2), *membre de l'organisation* (1) ; *trésorier* en rôle parallèle cumulable ; *visiteur* sans compte. Droits détaillés au tableau 7 de l'annexe C (p. 22).

Fonctionnalités principales. Festivals et zones (tarifaires, du plan), copie d'un festival ; structures et contacts ; catalogue de jeux ; suivi des exposants (workflow) ; réservations, prix et remises ; placement des jeux ; factures PDF et paiements. Détail des besoins (BF1 à BF20), règles et cas d'utilisation : annexe E (p. 23) et dans le document de besoin (`docs/besoin.tex`), qui fait référence ; réponses du client : tableau 9 (p. 25).

Hors périmètre. Bénévoles (environ 300, sans compte), staff de présentation des jeux, nombre de personnes par table, compensation des absents, envoi des factures par courriel (le trésorier envoie lui-même le PDF).

Équipe. Trois membres pour quatre rôles : la relation client est portée par toute l'équipe (tableau 1 (p. 2)) ; chacun code dans toutes les couches.

Tableau 1 – Responsables des rôles de l'équipe

Rôle	Responsable
Relation client et besoin	toute l'équipe
Architecture (garant du DAT)	Ines Mazgar
Déploiement (Docker, CI/CD, VM)	Mathieu Marty
Qualité (tests, revues, conventions)	Kim Etienne

2 Contraintes et exigences non fonctionnelles

Tableau 2 – Contraintes et exigences non fonctionnelles, et leur conséquence sur l'architecture

Exigence	Description	Conséquence sur l'architecture
Contraintes imposées	Angular ≥ 22 en syntaxe moderne (composants *standalone*, signaux, `inject()`, `@if` / `@for`) ; API Node.js et TypeScript (Express, sans framework applicatif) ; PostgreSQL via `pg`, sans ORM, requêtes paramétrées ; Docker Compose seul prérequis ; déploiement sur la VM par un pipeline GitLab CI/CD ; 10 semaines.	Trois briques (front, API, base) ; SQL dans une couche dédiée (ADR-005), couche d'accès aux données écrite à la main ; mêmes images en développement et en production (ADR-010).
Sécurité	Données commerciales et comptables sensibles ; six rôles ; factures immuables ; l'espace public n'expose aucune donnée financière ou personnelle (contacts…).	Droits vérifiés dans l'API (ADR-001, ADR-002) ; mots de passe hachés ; cookie HttpOnly (ADR-006) ; immuabilité en base (ADR-007).

1. CRM (*Customer Relationship Management*) : gestion de la relation client. 2. ADR (*Architecture Decision Record*) : fiche qui consigne une décision d'architecture, ses options et ses conséquences (section 9).

Tableau 2 – Contraintes et exigences non fonctionnelles (suite)

Exigence	Description	Conséquence sur l'architecture
Intégrité des données	Pas de surréservation ; facture jamais modifiée ni supprimée ; actions datées (Q20).	Transactions et verrous (ADR-008) ; contraintes et déclencheurs ; `timestamptz`[3].
Performance / volumétrie	Environ 1 000 visiteurs en lecture *[à confirmer : chiffre repris de la première version du DAT, à faire confirmer par le client]* ; quelques rédacteurs ; 307 structures et 1 445 jeux dans les données d'exemple fournies.	API sans état ; pool `pg` ; index ; pagination ; cache HTTP des routes publiques.
Disponibilité	Usage interne ; quelques minutes de coupure tolérées au déploiement *[à confirmer : tolérance du client]*.	Une seule VM ; sauvegarde quotidienne ; retour arrière rapide (ADR-010).
Maintenabilité	Équipe de trois, 10 semaines, reprise possible par une autre équipe.	Couches explicites, TypeScript strict, tests en CI, ADR versionnés.

3 Vue d'ensemble

L'architecture est décrite avec le modèle C4 de Simon Brown [4], par zooms successifs ; ses quatre niveaux sont résumés au tableau 8 de l'annexe D (p. 22). **Le niveau 1 (*contexte*)** montre le système comme une seule boîte entourée de ses utilisateurs ; il s'adresse à tous. **Le niveau 2 (*briques logicielles*, *containers* en C4)** montre les applications et les stockages qui le composent, avec leurs technologies et protocoles ; ces *containers* n'ont rien à voir avec Docker, traité en section 7. **Le niveau 3** est remplacé par la section 4 et **le niveau 4** (code) est laissé au dépôt.

3. TIMESTAMPTZ (*timestamp with time zone*) : type PostgreSQL qui stocke une date et une heure avec leur fuseau horaire. 4. `https://c4model.com`

3.1 Diagramme de contexte (C4 niveau 1)

La figure 1 (p. 4) montre le système et ses sept rôles. Aucun système externe n'est utilisé (ni courriel, ni paiement, ni fournisseur d'identité). Les exposants n'y figurent pas : ils n'utilisent pas l'application, et les échanges avec eux se font hors application, qui en garde la trace datée. GitLab et la VM [5] relèvent de l'infrastructure (sections 7 et 8).

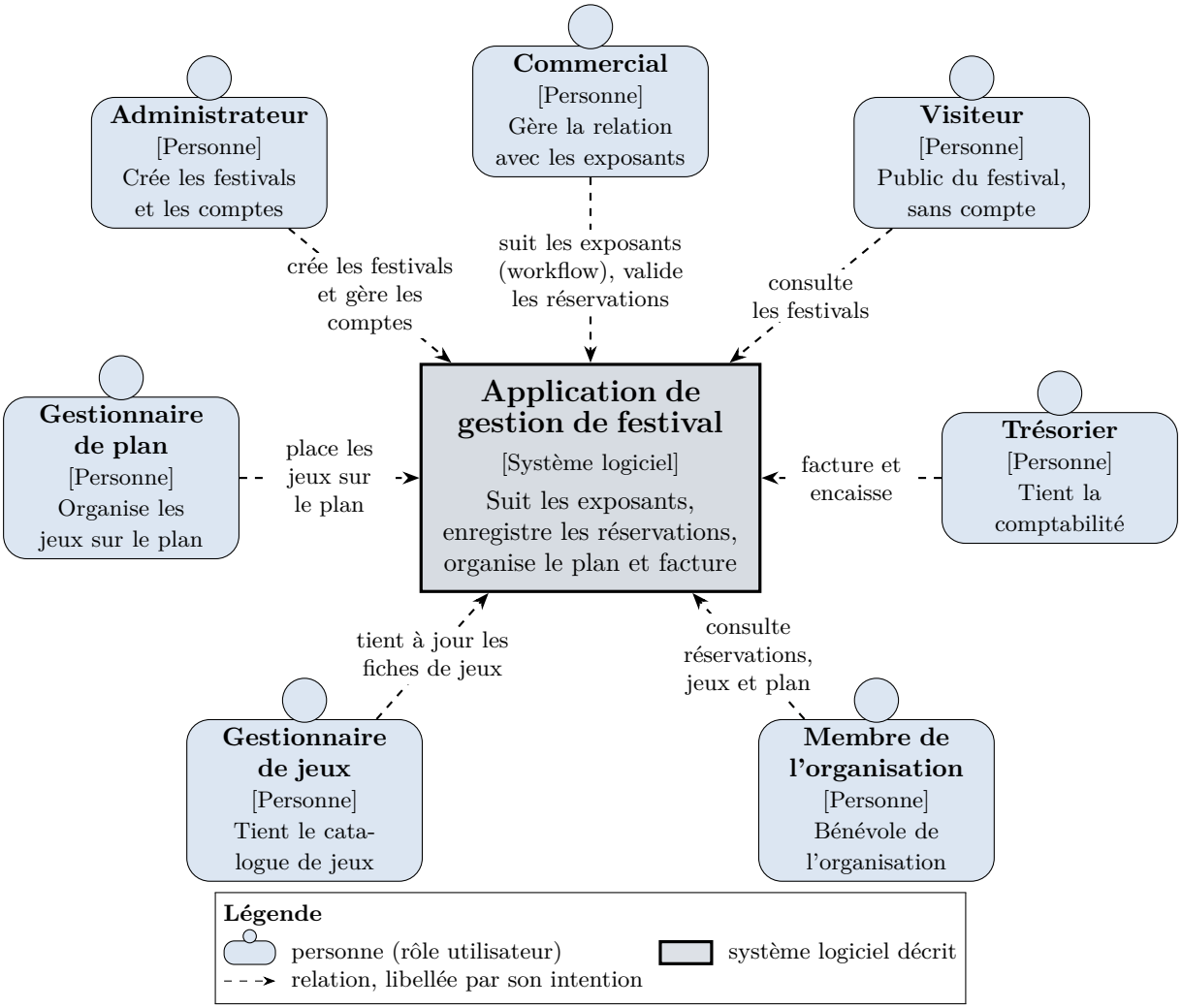


FIGURE 1 – [Diagramme de contexte, C4 niveau 1] Périmètre : l'application de gestion de festival. Aucun système logiciel externe n'interagit avec elle.

3.2 Diagramme des briques logicielles (C4 niveau 2)

La figure 2 (p. 5) ouvre la boîte du système. Le **serveur web** livre les fichiers produits par `ng build`. L'**application monopage** s'exécute dans le navigateur et ne parle qu'à l'API, en JSON [6]. L'**API** est la seule à accéder aux stockages ; elle porte les règles, les droits et la génération des PDF (PDFKit). La **base** garantit l'intégrité (contraintes, transactions, déclencheurs). Les **fichiers des règles** sont les PDF déposés par le gestionnaire de jeux ; la base n'en garde que le chemin, et les factures PDF ne sont pas stockées mais regénérées (ADR-012).

5. VM : machine virtuelle. 6. API (*Application Programming Interface*) : interface de programmation ; REST : style d'API fondé sur HTTP et des ressources nommées ; JSON : format texte d'échange de données.

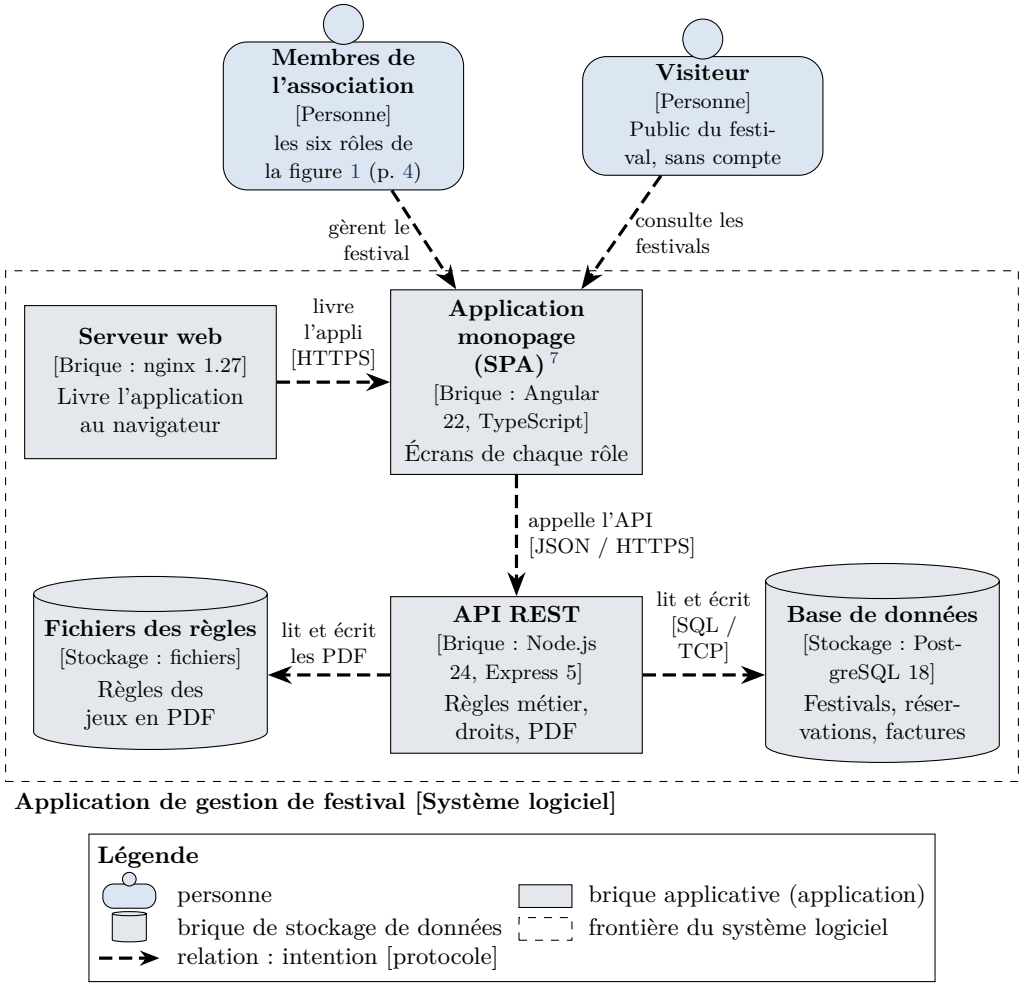


FIGURE 2 – [Diagramme des briques logicielles, C4 niveau 2] Périmètre : l'application de gestion de festival. Les personnes sont celles de la figure 1 (p. 4) (les six rôles internes, qui utilisent tous la même brique, sont regroupés). Aucun élément de déploiement (VM, conteneurs Docker, reverse proxy, ports) : voir la figure 5 (p. 11).

4 Architecture applicative

4.1 Front-end (Angular)

— **Organisation du code** : composants *standalone*, un dossier par fonctionnalité (`festivals`, `structures`, `jeux`, `suivi`, `reservations`, `facturation`, `comptes`) ; `core/` (authentification, intercepteurs, gardes) et `shared/` (composants réutilisables, par exemple `carte-festival` ou `carte-facturation`).

— **Routage** : fonctionnalités chargées à la demande (`loadChildren`) ; gardes `canMatch` selon le niveau du compte, pour l'ergonomie seulement : la sécurité est dans l'API.

— **Gestion de l'état** : chaque service expose l'état de sa ressource par des signaux en lecture seule, et par des `computed` pour les valeurs dérivées (tables restantes, reste à payer…) ; les composants les obtiennent par `inject()`. Exemples : `SessionService` (compte connecté), `FestivalCourantService` (par défaut le dernier festival créé). Pas de NgRx : l'état partagé est réduit.

— **Communication avec l'API** : `HttpClient` avec `withCredentials`, un service par ressource ; un intercepteur (fonction appelée automatiquement sur chaque requête et chaque

7. SPA (*Single Page Application*) : application web chargée une seule fois, qui met ensuite à jour la page sans la recharger.

réponse HTTP) redirige sur 401 et affiche les messages 403 et 409 [8].

4.2 Back-end

— **Technologie** : Node.js 24 LTS [9], TypeScript 5 strict, Express 5, `pg` 8, `zod`, `bcrypt`, `jsonwebtoken`, `pdfkit` ; versions figées dans `package-lock.json`.
— **Architecture en couches** (ADR-005) : *routes* (URL et méthode, intergiciels [10] de droits) → *contrôleurs* (lecture et validation de la requête, code HTTP) → *services* (règles métier, masquage, sans rien connaître de HTTP) → *dépôts* (seul code qui contient du SQL [11]). Chaque couche n'appelle que la couche inférieure.
— **Organisation des requêtes SQL** : une fonction par requête dans `src/depots` ; requêtes paramétrées (`$1`, `$2`), jamais de concaténation [12] ; opérations multi-tables dans `avecTransaction` [13], avec verrouillage des zones [14] (ADR-008) ; statut lu par la vue [15] `v_reservation_statut` (ADR-009).
— **Validation et erreurs** : schéma `zod` [16] appliqué par chaque contrôleur ; erreurs métier typées, traduites en JSON par un intergiciel unique ; erreurs 500 [17] journalisées sans détail exposé.

4.3 Conventions de l'API REST

Tableau 3 – Conventions de l'API REST

Élément	Convention retenue
Préfixe / versioning	`/api/v1`
Nommage des ressources	noms français au pluriel, en kebab-case (`/zones-tarifaires`) ; actions métier en `PATCH` sur une sous-ressource (`/validation`, `/annulation`)
Format des échanges	JSON en camelCase ; dates au format ISO 8601 (`2026-10-23` ou `2026-10-23T14:05:00Z`) ; montants en chaîne décimale (`"125.50"`) pour éviter les arrondis du type `number`
Format des erreurs	`{ "code": "CAPACITE_DEPASSEE", "message": "...", "details": {...} }`
Pagination / filtrage	`?page=1&taille=25&tri=nom&ordre=asc` ; filtres nommés (`?statut=discussion&role=editeur`) ; réponse `{ "elements": [...], "total": 312 }`

Le tableau 4 (p. 7) donne les principales ressources [18] ; la liste complète est au tableau 10 de l'annexe G (p. 26).

8. Codes HTTP : **401** : non authentifié (pas de jeton ou jeton expiré) ; **403** : authentifié mais droits insuffisants ; **409** : conflit avec l'état des données (capacité dépassée, facture déjà émise…). 9. LTS (*Long Term Support*) : version maintenue sur une longue durée. 10. Intergiciel (*middleware*) : fonction Express exécutée avant le contrôleur, par exemple pour vérifier les droits. 11. SQL : langage de requête des bases relationnelles ; HTTP : protocole d'échange du Web. 12. Requête paramétrée : les valeurs sont envoyées séparément du texte SQL, donc une saisie malveillante ne peut pas être exécutée comme du SQL (injection SQL). 13. Transaction : écritures appliquées en « tout ou rien » ; `avecTransaction` l'ouvre (`BEGIN`), la valide (`COMMIT`) ou l'annule (`ROLLBACK`). 14. Verrouillage : `SELECT ... FOR UPDATE` bloque la zone jusqu'à la fin de la transaction ; une réservation simultanée attend son tour. 15. Vue : requête SQL enregistrée, interrogée comme une table ; elle calcule ici « considéré absent » à la lecture. 16. `zod` : bibliothèque qui décrit la forme attendue d'une donnée (champs, types, bornes) et vérifie la requête reçue (sinon 400). 17. Codes HTTP : **500** : erreur interne du serveur. 18. Codes HTTP : **200** : réussite ; **201** : ressource créée ; **400** : requête invalide (données mal formées) ; **401** : non authentifié (pas de jeton ou jeton expiré) ; **403** : authentifié mais droits insuffisants ; **404** : ressource introuvable ; **409** : conflit avec l'état des données (capacité dépassée, facture déjà émise…) ; **429** : trop de requêtes (limitation de débit).

Tableau 4 – Principales ressources de l'API (rôle minimal et codes de réponse)

Méthode	Route	**Description**	**Rôle**	**Codes**
`POST`	`/api/v1/auth/connexion`	Connexion, pose le cookie	public	200, 400, 401, 429
`GET`	`/api/v1/festivals`	Liste des festivals (cartes)	public	200
`POST`	`/api/v1/festivals`	Création (ou copie avec `copieDe`)	admin	201, 400, 403, 409
`GET`	`/api/v1/festivals/:id/suivi`	Tableau de suivi des exposants	commercial	200, 401, 403
`POST`	`/api/v1/reservations`	Création d'une réservation	commercial	201, 400, 403, 409
`PATCH`	`/api/v1/reservations/:id/validation`	Validation (ADR-002)	commercial	200, 403, 409
`PUT`	`/api/v1/reservations/:id/jeux`	Jeux et placement sur le plan	gest. plan	200, 400, 403, 409
`POST`	`/api/v1/reservations/:id/factures`	Création d'une facture	trésorier	201, 403, 409
`GET`	`/api/v1/factures/:id/pdf`	Téléchargement du PDF	trésorier, com.	200, 403, 404
`POST`	`/api/v1/factures/:id/paiements`	Enregistrement d'un paiement	trésorier	201, 400, 403

5 Modèle de données

Le modèle relationnel est présenté seul, en format paysage, sur sa propre page (figure 3 (p. 8)).

Choix structurants. Une table unique `structure` évite de dupliquer une entreprise à la fois éditrice et exposante (ADR-011). Une `reservation` est le dossier d'un exposant pour un festival : elle existe dès le premier contact et porte le workflow, d'où l'unicité `(festival_id, exposant_id)`. Une facture fige ses lignes (ADR-007). Un compte porte un niveau entier et un indicateur trésorier (ADR-013) ; rôle des tables en annexe H (p. 28).

— **Représentation des données** : montants en `NUMERIC(10,2)` (calculs exacts) ; `DATE` pour les dates calendaires, `TIMESTAMPTZ` pour les actions datées ; états en `VARCHAR` contraints par `CHECK`, plus simples à faire évoluer qu'un `ENUM`.

— **Contraintes d'intégrité** : clés étrangères en `RESTRICT`[19] par défaut, `CASCADE`[20] pour les seules données dépendantes (contacts, lignes de réservation) ; unicités (nom de festival, zone par festival, numéro de facture) ; `CHECK`[21] sur les quantités, les dates et le non-cumul des remises *[à confirmer : Q19]* ; déclencheur d'immuabilité des factures.

— **Index** : toutes les clés étrangères ; `reservation(festival_id, statut)` pour le suivi ; noms des structures et des jeux.

— **Gestion du schéma** : migrations SQL[22] numérotées, appliquées au démarrage de l'API (table `schema_migrations`), uniquement additives (ADR-010) ; script d'import des données fournies, sans le lien type de jeu → zone.

6 Sécurité

— **Mot de passe** : il n'est jamais stocké. La base ne garde que son *haché* bcrypt, une empreinte à sens unique dont on ne peut pas retrouver le mot de passe ; bcrypt y ajoute un *sel* (un aléa propre à chaque compte) et il est volontairement lent, ce qui décourage les essais en masse.

— **Jeton** (ADR-006) : une fois connecté, l'utilisateur reçoit un JWT (*JSON Web Token*), un petit texte qui contient son identifiant, son niveau et l'indicateur trésorier, signé avec une clé secrète connue de la seule API (HS256). Toute modification casse la signature et le jeton est refusé. Il expire au bout de 8 h, sans renouvellement : il suffit de se reconnecter.

19. `ON DELETE RESTRICT` : la base refuse de supprimer une ligne tant que d'autres lignes y font référence (un festival qui a des réservations, par exemple). 20. `ON DELETE CASCADE` : supprimer une ligne supprime aussi automatiquement les lignes qui en dépendent (les contacts d'une structure supprimée, par exemple). 21. `CHECK` : condition vérifiée par la base à chaque écriture, par exemple `nb_tables > 0` ; une valeur qui ne la respecte pas est refusée. 22. Migration SQL : fichier numéroté (`001_initial.sql`, `002_ajout_repas.sql`…) qui fait évoluer la structure de la base d'une version à la suivante. Chaque migration n'est appliquée qu'une fois, dans l'ordre ; la table `schema_migrations` note celles déjà passées.

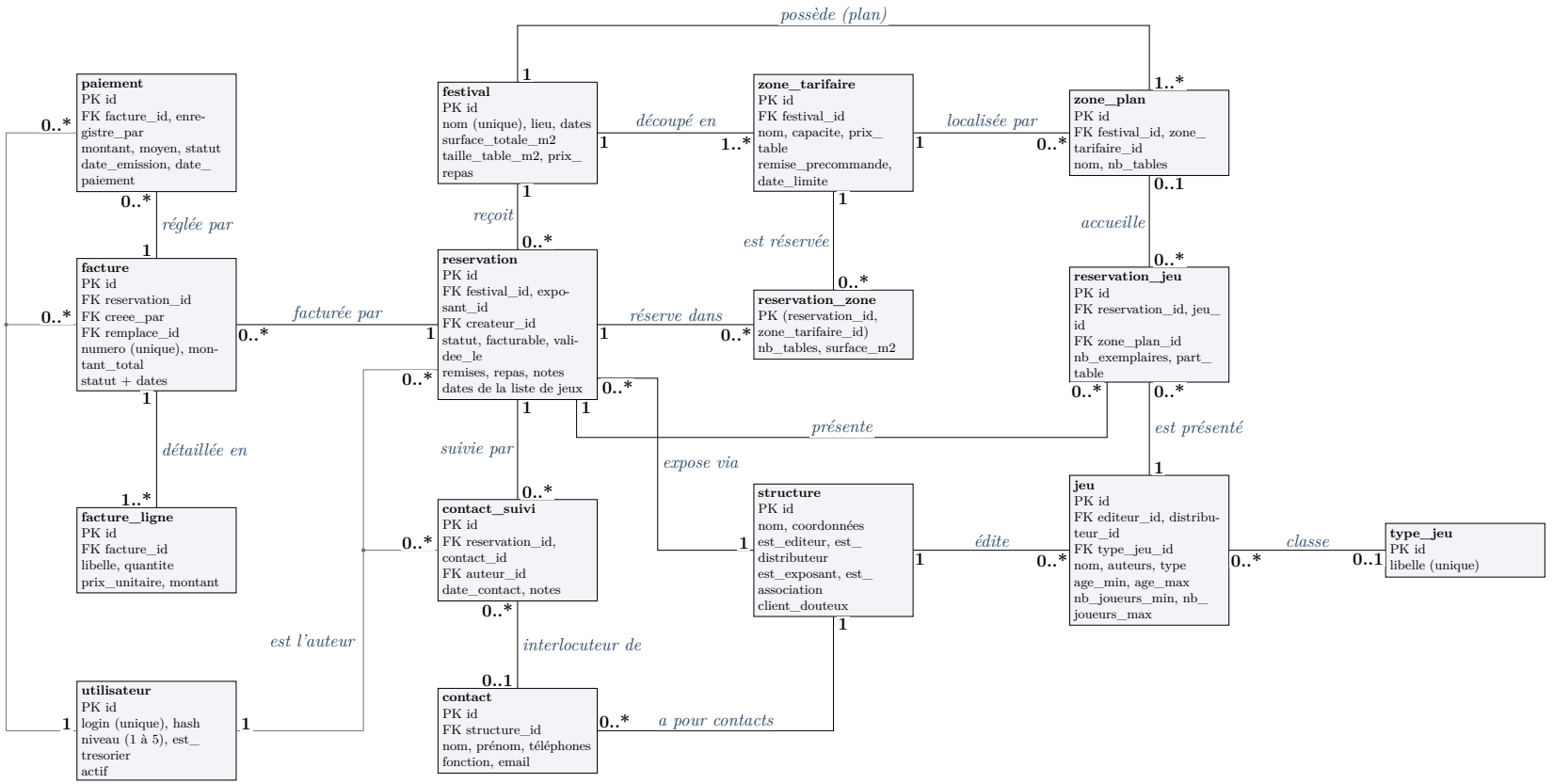


FIGURE 3 – [Modèle relationnel] Relations un-à-plusieurs, portées par une clé étrangère du côté « plusieurs » ; multiplicités UML aux deux extrémités. Les liens gris partent de `utilisateur` : ils indiquent quel compte a créé la réservation, émis la facture, saisi le paiement ou noté le suivi. Non représenté : `jeu.distributeur_id`. Le rôle de chaque table est expliqué en annexe H (p. 28). Script complet en annexe I (p. 29).

— **Cookie** : le jeton est rangé dans un cookie `HttpOnly` (le JavaScript de la page ne peut pas le lire), `Secure` (envoyé seulement en HTTPS) et `SameSite=Strict` (jamais envoyé depuis un autre site). Le front ne garde que le profil.
— **Autorisation** : à chaque requête, l'API vérifie le jeton puis le niveau exigé par la route (`exigerNiveau(n)`, `exigerRole('tresorier')`) ; les services masquent ensuite les informations commerciales (ADR-002). Cacher un bouton dans Angular ne protège rien.
— **Protections** : contre l'*injection SQL*, des requêtes paramétrées ; contre le *XSS* (un pirate fait exécuter son script dans la page d'un autre utilisateur), Angular affiche les données comme du texte et non comme du code, et la CSP (*Content Security Policy*) envoyée par nginx n'autorise que les scripts du site ; contre le *CSRF* (un site piégé envoie une requête avec votre cookie), `SameSite=Strict` et la même adresse pour le front et l'API, donc pas de CORS (autorisation donnée à un autre site d'appeler l'API) ; `helmet`, qui ajoute en une ligne les en-têtes de sécurité usuels d'Express ; au plus 10 essais de connexion par minute et par IP ; PDF limités en taille et en type.
— **Secrets** : mots de passe de la base et clé du JWT dans des variables d'environnement, jamais dans Git ; `.env` local ignoré (seul `.env.example` est versionné) ; variables protégées de GitLab en production.
— **Transport** : HTTPS chiffre les échanges entre le navigateur et nginx ; personne sur le réseau ne peut lire le mot de passe ou le cookie. La VM n'étant joignable que depuis Polytech ou par VPN (réseau privé virtuel de l'école), Let's Encrypt n'est pas utilisable *[à confirmer : certificat de l'école ou auto-signé]*. Sans HTTPS, l'option `Secure` bloquerait le cookie et la connexion.

Matrice des droits. Voir le tableau 7 de l'annexe C (p. 22) ; c'est lui que vérifient les tests rôle × route de la CI.

Déroulement d'une connexion. La figure 4 (p. 10) suit une connexion. L'application envoie login et mot de passe à `POST /api/v1/auth/connexion` (étapes 1–2). L'API lit le compte (3–4), *vérifie elle-même le mot de passe* avec bcrypt, calcule le niveau et renvoie un JWT de 8 h dans un cookie `HttpOnly` (5–6) : le front n'y a pas accès et ne garde que le profil (7). Le cookie accompagne ensuite chaque requête ; l'API vérifie signature, expiration et niveau requis (8–11). En cas d'échec, un mauvais mot de passe et un login inconnu donnent le même 401 [23], pour ne pas révéler les comptes (12–13) ; un jeton expiré donne 401 et renvoie vers la connexion (14–16).

7 Architecture de déploiement

La figure 5 (p. 11) place chaque brique de la figure 2 (p. 5) sur la VM du groupe (`162.38.111.31`, réseau de Polytech ou VPN), où chaque membre a un compte. Docker y est *rootless* (démon d'un utilisateur, sans droits administrateur) : l'application tourne sous le seul compte du responsable du déploiement, `mathieu.marty`.

Tableau 5 – Machine de déploiement : rôle, services et ports

VM	Rôle	Conteneurs / services	Ports exposés	Accessible depuis
`162.38.111.31`	Production unique	`front`, `api`, `db` sous le Docker rootless de `mathieu.marty` ; `crontab` de sauvegarde	22 ; HTTPS de `front` *[à confirmer : 80/443 ou ports hauts en Docker rootless]*	Réseau Polytech ou VPN ; runner GitLab (22, clé dédiée)

23. Codes HTTP : **200** : réussite ; **401** : non authentifié (pas de jeton ou jeton expiré).

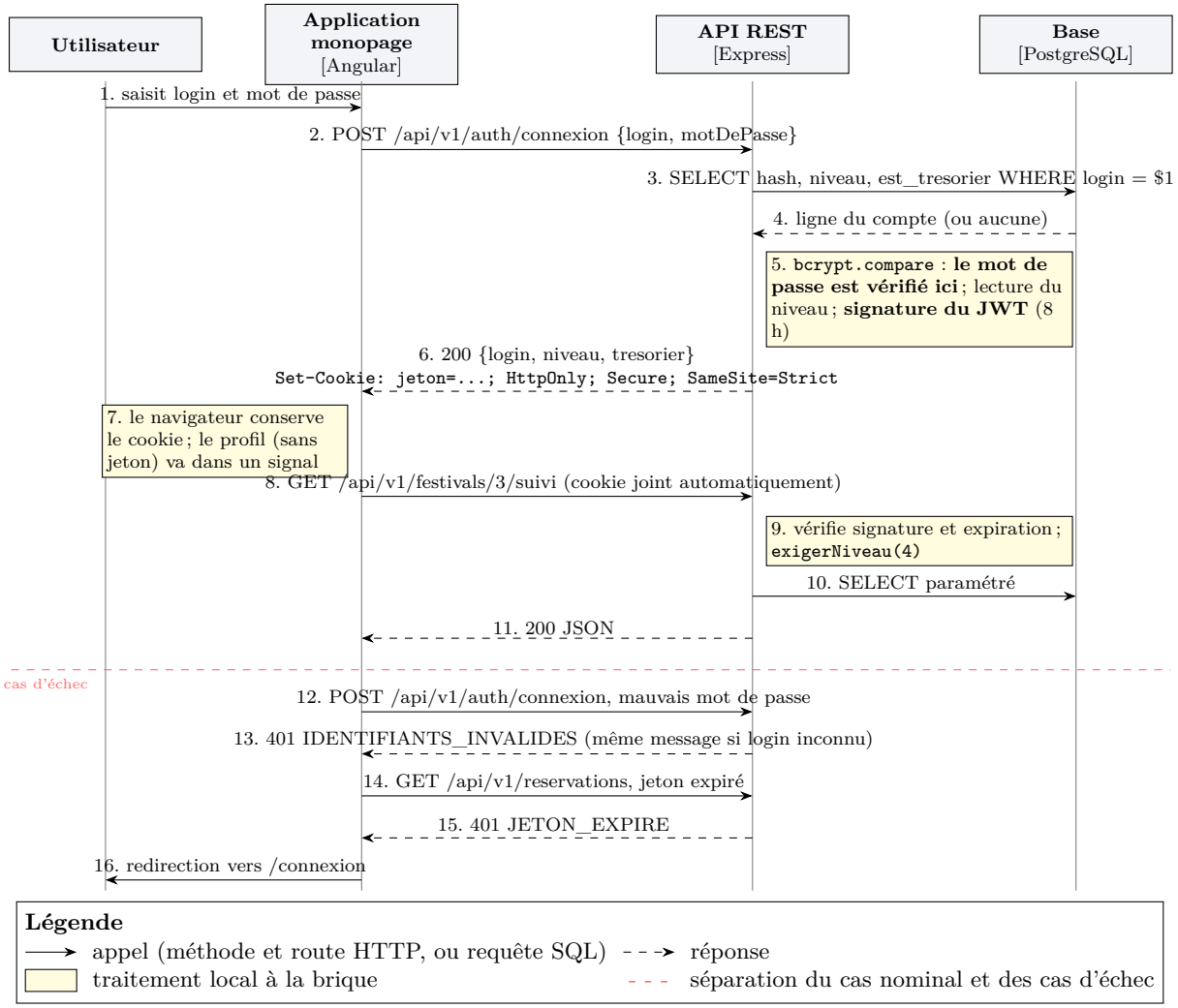
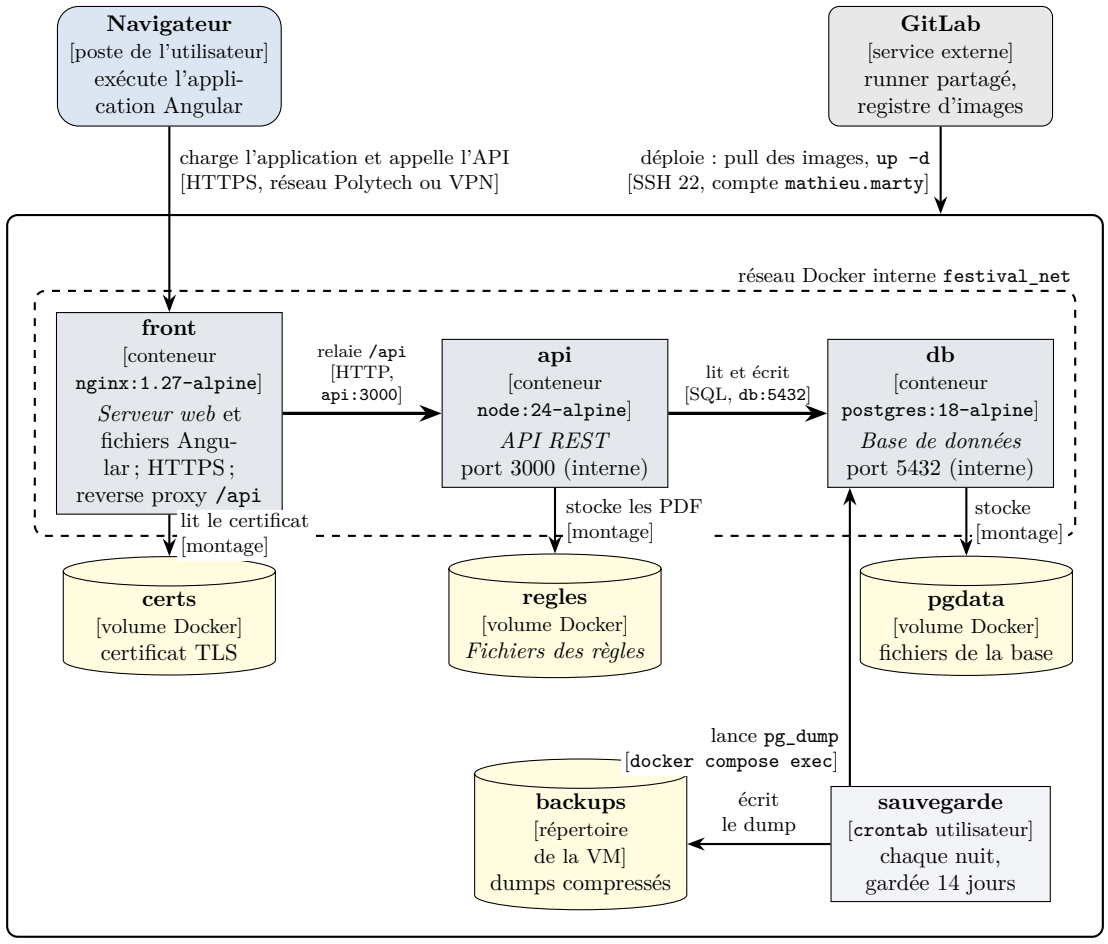


FIGURE 4 – [Diagramme de séquence] [24]Authentification puis requête authentifiée ; participants : les briques de la figure 2 (p. 5).

— **Conteneurisation** : Docker rootless (installé par `dockerd-rootless-setuptool.sh install`) sous le compte `mathieu.marty`, avec `loginctl enable-linger` pour qu'il survive à la déconnexion *[à confirmer : linger autorisé]* ; trois images (`front`, `api`, `postgres`), Dockerfiles multi-étapes ; un réseau interne ; volumes `pgdata`, `regles` et `certs`, stockés dans `~/.local/share/docker`.
— **Environnements** : `docker-compose.yml` (développement) construit les images, monte le code, lance `ng serve` et publie 4200, 3000 et 5432 en local ; `docker-compose.prod.yml` utilise les images du registre et ne publie que le port HTTPS de `front`.
— **Persistance et sauvegarde** : sauvegarde quotidienne de la base de données à l'aide de `pg_dump` (outil fourni avec PostgreSQL permettant d'exporter le contenu de la base à un instant donné), ainsi qu'archivage du volume `regles`. Les sauvegardes sont conservées pendant 14 jours. En cas de perte ou de corruption de la base, la restauration s'effectue à l'aide de `pg_restore`, procédure testée avant la soutenance.
— **Journalisation** : consultation des journaux des conteneurs via `docker compose logs`, rotation des journaux Docker avec le pilote `json-file` et journalisation des accès Nginx. Aucune solution de supervision dédiée n'est mise en place.

24. Codes HTTP : **200** : réussite ; **401** : non authentifié (pas de jeton ou jeton expiré).



VM du groupe 11, `162.38.111.31` [Linux, Docker rootless, docker compose]

FIGURE 5 – [Diagramme de déploiement] Périmètre : la VM de production, joignable depuis le réseau de Polytech ou par VPN. Seuls SSH (22) et le port HTTPS de `front` sont exposés *[à confirmer : 80/443 ou ports hauts en Docker rootless]* ; 3000 et 5432 restent internes au réseau Docker ; HTTPS est terminé dans `front`.

8 Build et déploiement (CI/CD)

— **Outil et fichier** : GitLab CI [25], `.gitlab-ci.yml` à la racine.
— **Déclencheurs** : toute branche et merge request (lint, test) ; `main`, protégée (build, deploy) ; tags `dat-v*` sans job applicatif.
— **Étapes** : *lint* (ESLint, typage) ; *test* (unitaires et API : droits, non-fuite, capacité concurrente, immuabilité, sur un PostgreSQL du pipeline ; couverture minimale 70 % *[à confirmer : seuil]*) ; *build* (images) ; *deploy*, qui échoue si `GET /api/v1/sante` ne répond pas 200 [26] en 60 s. Tout échec arrête le pipeline.
— **Artefacts et registre** : images `api` et `front` étiquetées par le SHA [27] court ; rapports de tests gardés 7 jours.
— **Accès à la VM** : compte `mathieu.marty` (pas de `sudo`, donc pas d'utilisateur dédié) ; une

25. CI/CD : intégration et déploiement continus, c'est-à-dire tests et mise en production automatiques à chaque modification. 26. Codes HTTP : **200** : réussite. 27. SHA : empreinte qui identifie un commit Git.

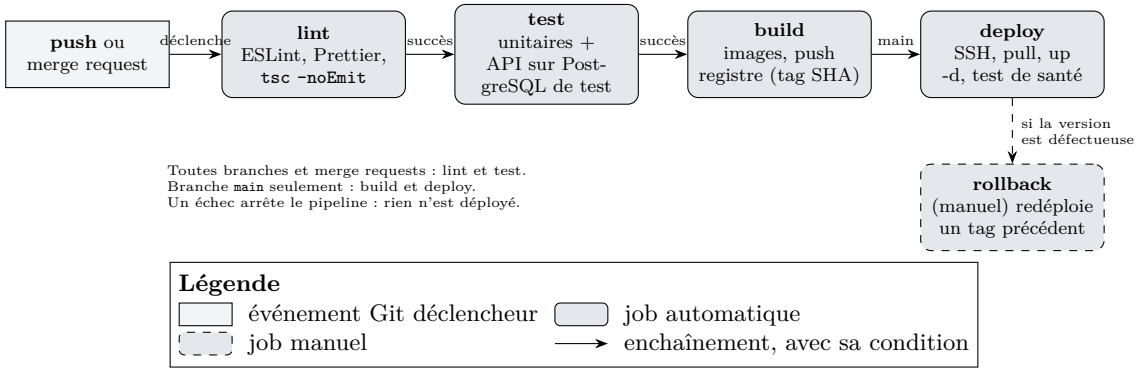


FIGURE 6 – [Pipeline CI/CD] Périmètre : `.gitlab-ci.yml`. Étapes dans l'ordre ; environnement cible unique : la production sur la VM.

clé SSH[28] réservée à la CI, ajoutée à son `authorized_keys` et stockée dans une variable GitLab de type fichier ; l'empreinte de la VM est enregistrée dans `known_hosts`, la liste des serveurs reconnus : le runner vérifie ainsi qu'il parle bien à notre VM, et non à une machine qui se ferait passer pour elle. Le runner doit pouvoir joindre la VM *[à confirmer : runner sur le réseau de Polytech]*.

— **Secrets** : variables protégées et masquées (`SSH_KEY`, `JWT_SECRET`, `POSTGRES_PASSWORD`, jeton du registre), écrites dans le `.env` de la VM par *deploy*.

— **Configuration par environnement** : `.env.example` versionné ; `.env` local en développement, généré sur la VM en production.

— **Retour arrière** : job manuel *rollback* avec `TAG=<sha précédent>` (`pull` puis `up -d`) ; les migrations étant additives, l'ancienne version fonctionne sur le nouveau schéma ; sinon, restauration de la sauvegarde.

9 Décisions d'architecture (ADR)

Le tableau 6 (p. 12) résume les décisions. Les fiches complètes, au format contexte, critères, options, décision et conséquences, sont en annexe A (p. 13) ; elles sont aussi versionnées dans `docs/adr/`.

TABLEAU 6 – Registre des décisions d'architecture

ADR	Décision	Statut	Date
ADR-001	Rôles hiérarchiques à niveaux et rôle trésorier parallèle	Acceptée	14/10/2026
ADR-002	Validation de la réservation et masquage en liste blanche	Acceptée	14/10/2026
ADR-003	Facturation réservée au trésorier, facture immuable	Acceptée	14/10/2026
ADR-004	Numéro retiré : correction d'erreur tracée dans le journal	Retiré	23/10/2026
ADR-005	Back-end en quatre couches (routes, contrôleurs, services, dépôts SQL)	Acceptée	24/10/2026
ADR-006	Authentification par JWT dans un cookie HttpOnly	Acceptée	23/10/2026
ADR-007	Facture : instantané des lignes, immuabilité garantie en base	Acceptée	23/10/2026
ADR-008	Contrôle de capacité par verrouillage transactionnel	Acceptée	23/10/2026
ADR-009	Statut « considéré absent » calculé à la lecture	Acceptée	24/10/2026
ADR-010	Déploiement par images versionnées, Docker rootless sur la VM	Acceptée	24/10/2026
ADR-011	Table unique `structure` ; la réservation porte l'exposant	Acceptée	24/10/2026
ADR-012	Stockage des PDF : règles en fichiers, factures regénérées	Acceptée	24/10/2026
ADR-013	Niveau et indicateur trésorier stockés dans le compte	Acceptée	24/10/2026

10 Limites et évolutions

— **Limites connues** : une seule VM ; jetons non révocables avant expiration (ADR-006) ; numérotation des factures *[à confirmer : continuité exigée ?]*.

— **Passage à l'échelle (×100)** : API dupliquée, CDN, réplica PostgreSQL.

28. SSH : protocole de connexion à distance chiffrée.

Annexes

Table des figures

1 Diagramme de contexte (C4 niveau 1) . . . 4
2 Diagramme des briques logicielles (C4 niveau 2) . . . 5
3 Modèle relationnel . . . 8
4 Diagramme de séquence de l'authentification . . . 10
5 Diagramme de déploiement . . . 11
6 Pipeline CI/CD . . . 12
7 Diagramme de cas d'utilisation (UML) . . . 25

Liste des tableaux

1 Responsables des rôles de l'équipe . . . 2
2 Contraintes et exigences non fonctionnelles, et leur conséquence sur l'architecture . 2
3 Conventions de l'API REST . . . 6
4 Principales ressources de l'API (rôle minimal et codes de réponse) . . . 7
5 Machine de déploiement : rôle, services et ports . . . 9
6 Registre des décisions d'architecture . . . 12
7 Matrice des droits par rôle . . . 22
8 Les quatre niveaux du modèle C4 et leur usage dans ce DAT . . . 22
9 Questions posées au client et réponses (entrevues du 07/10 et du 23/10/2026) . . . 25
10 Liste complète des endpoints de l'API . . . 27
11 Rôle des tables qui peuvent prêter à confusion . . . 28
12 Journal des évolutions du DAT . . . 32

A Fiches ADR

ADR-001 - Rôles hiérarchiques à niveaux et rôle trésorier parallèle

Statut : Acceptée **Date :** 14/10/2026

Contexte

Le brouillon prévoyait six rôles cumulables et indépendants (admin, organisateur, membre de l'organisation, gérant de table, trésorier, visiteur). Le client décrit des rôles « descendants » : chaque rôle inclut les droits des rôles inférieurs, les noms sont libres, plusieurs comptes peuvent partager un rôle, et le trésorier est à part. Le mot « organisateur » désignait à la fois la structure qui réserve et un rôle utilisateur.

Critères de décision (fixés avant la comparaison)

(1) Fidélité au modèle d'inclusion décrit par le client ; (2) vérification simple et uniforme dans l'API ; (3) possibilité de tester automatiquement toute la matrice rôle × route ; (4) coût d'évolution si le client ajoute un rôle.

Options considérées

A. Rôles indépendants cumulables, chaque route listant les rôles autorisés : souple, mais l'inclusion doit être répétée sur chaque route, et un oubli crée une faille ou un blocage (critères 1 et 2 mal couverts).

B. Niveaux entiers ordonnés dans le code (`admin`=5 … `membre_orga`=1) et rôle `tresorier` hors hiérarchie : une seule comparaison par route, inclusion garantie par construction ; un droit « en diagonale » n'est pas exprimable.

Décision

Option B. Niveau effectif d'un compte = maximum de ses rôles hiérarchiques ; le trésorier s'ajoute sans changer ce niveau. Le rôle utilisateur « organisateur » devient « commercial » ; « organisateur » n'est plus utilisé (la structure qui réserve est l'*exposant*, ADR-011). Le

dernier administrateur ne peut être ni supprimé ni rétrogradé. Le visiteur consulte sans compte (confirmé par le client, Q26).

Conséquences

Positives : intergiciels `exigerNiveau(n)` et `exigerRole('tresorier')` ; matrice testable en CI ; vocabulaire clarifié. *Négatives* : ajouter un rôle intermédiaire oblige à renuméroter les niveaux et à relire toutes les routes ; un droit transverse (par exemple un membre qui saisirait des jeux sans voir le plan) demandera un nouvel ADR ; le niveau étant embarqué dans le jeton (ADR-006), un changement de rôle ne prend effet qu'à la reconnexion. Le rôle « gérant de table » disparaît, donc `reservation_jeu.gerant_id` est supprimé.

Note du 24/10/2026 : le stockage du niveau est précisé par l'ADR-013 (une colonne `niveau` et un indicateur `est_tresorier` dans `utilisateur`) ; le principe de cette décision est inchangé.

ADR-002 - Validation de la réservation et masquage des informations commerciales en liste blanche

Statut : Acceptée **Date :** 14/10/2026

Contexte

À partir du gestionnaire de plan, les rôles ne doivent voir que la réservation finale : ni remises, ni prix, ni notes, ni statut du workflow, ni contacts de suivi. Les jeux sont ajoutés bien après la négociation, une fois les tables arrêtées.

Critères de décision (fixés avant la comparaison)

(1) Aucune fuite possible, y compris en appelant l'API directement ; (2) la règle est appliquée à un seul endroit ; (3) testabilité ; (4) un champ ajouté plus tard ne doit pas fuiter par défaut.

Options considérées

A. Masquage dans le front : les données transitent quand même sur le réseau (critère 1 violé). Rejetée d'emblée.

B. Vues SQL distinctes (`v_reservation_publique`) : garantie au niveau SQL, mais la règle de droits se répartit entre SQL et code, et chaque requête de liste existe en double.

C. Liste noire : retirer les champs interdits dans le service. On part de la réservation complète et on efface ce qui est commercial (`delete r.remise`, `delete r.notes`…). Simple, mais si l'on ajoute plus tard un champ commercial et qu'on oublie de l'effacer, il part vers tous les rôles (critère 4 violé).

D. Liste blanche : ne recopier que les champs autorisés dans le service. Une *projection*, c'est le choix des champs que l'on renvoie. Pour les rôles qui n'ont pas droit aux informations commerciales, le service fabrique une nouvelle réponse en recopiant un à un les seuls champs permis, par exemple `{ id, exposant, tablesParZone, jeux, zonesDuPlan }` ; remises, prix, notes, statut et contacts de suivi n'y figurent tout simplement pas.

Décision

Option D : on écrit la liste de ce qui est *permis* plutôt que celle de ce qui est interdit. Ainsi, un champ ajouté plus tard est caché par défaut : en cas d'oubli, il manque à l'écran (gênant, mais sans danger) au lieu d'être divulgué. Une colonne `validee_le` est ajoutée à la réservation, renseignée par `PATCH /reservations/:id/validation` (commercial). Avant validation, une réservation est invisible pour les niveaux inférieurs à 4 (réponse 404 [29], pour ne pas révéler son existence). L'ajout de jeux n'est possible qu'après validation.

Conséquences

Positives : un seul point de contrôle, nouveau champ masqué par défaut, test de non-fuite en CI. *Négatives* : deux types TypeScript (`ReservationComplete`, `ReservationPublique`) à maintenir ; la modification d'une réservation déjà validée n'est pas encore définie *[à confirmer : faut-il la dé-valider ? que deviennent les jeux placés ?]*.

29. Codes HTTP : **404** : ressource introuvable.

ADR-003 - Facturation réservée au trésorier, facture immuable

Statut : Acceptée **Date :** 14/10/2026

Contexte

Le brouillon faisait émettre les factures par l'organisateur et valider par le trésorier. Le client précise que le trésorier crée la facture à partir de la réservation, qu'elle « ne bouge plus », qu'il peut l'annuler, la rembourser et noter son paiement, et qu'un ajout de tables ou de repas produit une nouvelle facture.

Critères de décision (fixés avant la comparaison)

(1) Intégrité comptable : un document envoyé ne change jamais ; (2) traçabilité datée de chaque action ; (3) séparation entre négociation (commercial) et facturation (trésorier) ; (4) simplicité pour le trésorier.

Options considérées

A. Facture recalculée à la volée depuis la réservation : simple, mais une modification de la réservation modifie rétroactivement un document déjà envoyé (critère 1 violé).

B. Facture modifiable tant qu'elle n'est pas payée : moins de factures, mais aucun historique fiable et contraire à la demande du client.

C. Facture immuable, créée uniquement par le trésorier ; toute correction passe par une annulation suivie d'une nouvelle facture.

Décision

Option C. Statuts `emise`, `annulee`, `a_rembourser`, `remboursee`, chacun daté ; « payée » est dérivé des paiements (ADR-007). Endpoints `POST /reservations/:id/factures`, `PATCH /factures/:id/annulation`, `PATCH /factures/:id/remboursement`. La façon dont le commercial signale un ajout au trésorier reste à préciser.

Conséquences

Positives : historique complet, droits simples à tester. *Négatives* : plusieurs factures par réservation, donc un reste à payer calculé sur l'ensemble ; le trésorier doit savoir qu'il faut refacturer, d'où un indicateur « réservation modifiée depuis la dernière facture » dans le suivi ; le sort d'un paiement partiel sur une facture annulée reste à définir *[à confirmer : report du paiement sur la nouvelle facture ?]*.

Note du 23/10/2026 (entrevue, Q20) : le signalement se fait en interne, hors application, mais l'application doit dater chaque action. Cela confirme la décision sans la modifier : chaque statut de facture porte sa date.

ADR-004 - Numéro retiré

Statut : Retiré (ce n'était pas une décision) **Date :** 23/10/2026

Ce qui s'est passé

Le 14/10/2026, l'équipe avait créé une fiche ADR-004 intitulée « Correction du schéma des entités ». Le diagramme de l'époque comportait une erreur : il reliait une réservation à une seule facture, et les paiements directement à la réservation. Or le texte et le SQL prévoyaient déjà plusieurs factures par réservation et des paiements rattachés à une facture. La fiche corrigeait donc le dessin.

Pourquoi ce n'est pas un ADR

Un ADR consigne un *choix* entre plusieurs options, avec des critères et des conséquences. Ici, il n'y avait rien à choisir : le diagramme était faux et il fallait le corriger. Une correction d'erreur se note dans le journal des évolutions du DAT, comme le prévoit la trame, et c'est là qu'elle figure désormais.

Pourquoi garder le numéro

Le numéro ADR-004 a déjà été utilisé, dans le dépôt et dans la version `dat-v1`. Un numéro d'ADR n'est jamais réutilisé, pour que chaque référence désigne toujours la même fiche : on garde donc cette fiche « retirée », et la décision suivante porte le numéro ADR-005.

ADR-005 - Back-end en quatre couches : routes, contrôleurs, services, dépôts SQL

Statut : Acceptée **Date :** 24/10/2026 (proposée le 23/10/2026)

Contexte

Express n'impose aucune organisation. Le SQL est écrit à la main (pas d'ORM). Les règles métier sont denses (prix, remises, capacité, masquage, workflow). L'équipe compte trois personnes qui codent toutes dans toutes les couches et doivent pouvoir expliquer n'importe quel fichier en soutenance.

Critères de décision (fixés avant la comparaison)

(1) Tester les règles métier sans base ni HTTP ; (2) concentrer le SQL pour relire facilement le paramétrage des requêtes ; (3) une seule responsabilité par couche, facile à expliquer à un nouvel arrivant ; (4) coût de mise en place.

Options considérées

A. Routes Express contenant SQL et logique : démarrage rapide, mais règles dupliquées entre routes, SQL dispersé, tests uniquement de bout en bout (critères 1, 2 et 3 violés).

B. Trois couches : routes, services, dépôts ; la route déclare l'URL *et* traite la requête (validation, appel du service, code HTTP). Moins de fichiers, mais la route mélange deux responsabilités, et le traitement de la requête ne se teste pas sans Express (critère 3 moins bien couvert).

C. Quatre couches : *routes* (associent une URL et une méthode HTTP aux intergiciels de droits et au contrôleur), *contrôleurs* (lisent la requête, la valident avec `zod`, appellent le service, choisissent le code HTTP), *services* (règles métier, sans rien connaître de HTTP), *dépôts* (seuls à contenir du SQL). C'est aussi l'organisation proposée en exemple par la trame du DAT.

Décision

Option C, pour qu'à chaque couche corresponde une seule responsabilité (critère 3), pour un coût faible : un fichier de plus par ressource par rapport à l'option B. Chaque couche n'appelle que la couche inférieure. Une fonction `avecTransaction(async (client) => ...)` ouvre la transaction dans le service et passe le même client `pg` à tous les dépôts concernés.

Conséquences

Positives : fichiers de routes lisibles d'un coup d'œil (la liste des URL de l'API) ; contrôleurs testables sans démarrer Express ; services testables avec de faux dépôts ; SQL relu en un seul endroit (`src/depots`). *Négatives* : cinq fichiers par ressource (route, contrôleur, service, dépôt, types) ; pour les ressources simples, contrôleur et service ne font souvent que relayer, ce que l'on accepte ; les dépôts doivent accepter un client `pg` en paramètre pour participer à une transaction.

ADR-006 - Authentification par jeton JWT dans un cookie HttpOnly

Statut : Proposée **Date :** 23/10/2026

Contexte

Connexion par login et mot de passe. L'API doit rester sans état pour pouvoir être dupliquée (environ 1 000 lecteurs). Le front et l'API sont servis sous la même origine grâce au reverse proxy. Les menaces principales sont le vol de jeton par XSS et la falsification de requête (CSRF).

Critères de décision (fixés avant la comparaison)

(1) Résistance au XSS ; (2) résistance au CSRF ; (3) API sans état ; (4) simplicité côté Angular ; (5) possibilité de révoquer un accès.

Options considérées

A. Session serveur (cookie opaque, sessions dans PostgreSQL) : révocation immédiate (5), mais état côté serveur et lecture en base à chaque requête (3).

B. JWT dans `localStorage`, envoyé dans l'en-tête `Authorization` : simple, insensible au CSRF, mais lisible par tout script injecté (1 violé).

C. JWT signé (HS256) dans un cookie `HttpOnly; Secure; SameSite=Strict` : illisible par le JavaScript, CSRF neutralisé par `SameSite=Strict` et la même origine, API sans état.

Décision

Option C. Durée de vie 8 h, sans jeton de rafraîchissement ; à l'expiration, l'utilisateur se reconnecte. Le jeton contient l'identifiant, le niveau effectif et l'indicateur trésorier. Mots de passe hachés avec bcrypt. Limitation de débit sur `/auth/login`.

Conséquences

Positives : aucun jeton accessible au JavaScript ; aucune lecture en base pour authentifier. *Négatives* : pas de révocation avant expiration : un compte supprimé ou rétrogradé reste actif jusqu'à 8 h (atténuation possible : vérifier `utilisateur.actif` à chaque requête, au prix d'une lecture indexée, à trancher) ; HTTPS obligatoire en production pour l'attribut `Secure` ; les tests d'API doivent gérer les cookies.

ADR-007 - Facture : instantané des lignes et immuabilité garantie en base

Statut : Proposée **Date :** 23/10/2026

Contexte

L'ADR-003 impose l'immuabilité, mais le schéma ne stockait que `montant_total` : le PDF aurait été régénéré à partir de la réservation, qui, elle, continue d'évoluer. Le PDF d'une ancienne facture aurait donc changé.

Critères de décision (fixés avant la comparaison)

(1) Reproduire à l'identique une facture déjà envoyée ; (2) immuabilité garantie même en cas de bogue applicatif ; (3) données interrogeables (liste des factures, reste à payer) ; (4) simplicité de sauvegarde.

Options considérées

A. Montant seul et PDF régénéré depuis la réservation : critère 1 violé.

B. Stocker le fichier PDF (volume ou `bytea`) : reproductible, mais binaires à sauvegarder à part et contenu non interrogeable (3, 4).

C. Stocker l'instantané : lignes figées (`facture_ligne` : libellé, quantité, prix unitaire, montant) et coordonnées de l'exposant figées ; PDF généré à la demande depuis cet instantané ; déclencheur PostgreSQL qui refuse toute suppression et toute modification autre que le statut et ses dates.

Décision

Option C. La création de la facture et de ses lignes se fait dans une transaction. Le statut « payée » n'est pas stocké : il est dérivé (somme des paiements encaissés $\geq$ montant), ce qui évite deux sources de vérité.

Conséquences

Positives : un PDF envoyé est toujours reproductible ; un bogue applicatif ne peut pas réécrire une facture. *Négatives* : duplication volontaire de données ; logique en base (déclencheur) à documenter et à tester ; une évolution du gabarit PDF change l'aspect, mais pas le contenu, des anciennes factures.

ADR-008 - Contrôle de capacité par verrouillage transactionnel des zones

Statut : Proposée **Date :** 23/10/2026

Contexte

Plusieurs commerciaux travaillent en même temps (Q24). Une réservation est refusée si elle dépasse la capacité de la zone ou l'espace restant du festival. Une vérification « lire puis écrire » laisse passer deux réservations simultanées qui lisent chacune deux tables libres et en prennent deux.

Critères de décision (fixés avant la comparaison)

(1) Aucune surréservation, même en concurrence ; (2) message d'erreur clair ; (3) simplicité d'explication ; (4) performance (les écritures sont rares).

Options considérées

A. Vérification applicative sans verrou : condition de course (1 violé).

B. Isolation `SERIALIZABLE` : correcte, mais erreurs de sérialisation à rejouer et comportement difficile à expliquer (3).

C. `SELECT ... FOR UPDATE` sur la ligne du festival puis sur les zones concernées, dans la transaction, avant recalcul et écriture.

Décision

Option C, dans `avecTransaction` ; verrous pris dans un ordre fixe (festival, puis zones par identifiant croissant) pour éviter les interblocages ; réponse `409 Conflict` [30] avec le nombre de tables restantes en cas de dépassement.

Conséquences

Positives : surréservation impossible, logique lisible. *Négatives* : les écritures sur un même festival sont sérialisées (négligeable au volume attendu) ; la règle vit dans le service et non dans le schéma, d'où un test d'intégration concurrent obligatoire dans la CI.

ADR-009 - Statut « considéré absent » calculé à la lecture

Statut : Acceptée **Date :** 24/10/2026 (proposée le 23/10/2026)

Contexte

Le client veut que les exposants sans réponse passent automatiquement à « considéré absent » à la date du festival (Q11). L'API pourra être dupliquée (section 2).

Critères de décision (fixés avant la comparaison)

(1) Aucun passage manqué (API arrêtée à minuit, redémarrage) ; (2) compatible avec plusieurs instances de l'API ; (3) simplicité ; (4) filtres et tris cohérents.

Options considérées

A. Tâche planifiée dans l'API (`node-cron`) : simple, mais exécutée par chaque instance (2) et manquée si l'API est arrêtée (1).

B. Extension `pg_cron` : fiable, mais extension à ajouter à l'image PostgreSQL et logique cachée en base.

C. Calcul à la lecture dans une vue : `CASE WHEN f.date_debut <= CURRENT_DATE AND r.statut IN ('pas_contacte','contacte','discussion') THEN 'considere_absent' ELSE r.statut END`.

Décision

Option C, vue `v_reservation_statut` utilisée par toutes les lectures. Le statut reste saisissable manuellement avant la date (les notes d'entrevue décrivent un passage manuel « pour ne plus l'avoir dans la liste ») *[à confirmer : statuts concernés par le passage automatique]*.

Conséquences

Positives : toujours exact, rien à planifier. *Négatives* : le statut stocké diffère du statut affiché, toute lecture doit passer par la vue ; la date du passage n'est pas enregistrée (c'est la date du festival).

ADR-010 - Déploiement par images versionnées, Docker rootless sur la VM

Statut : Acceptée **Date :** 24/10/2026 (proposée le 23/10, révisée le 24/10/2026)

Contexte

Une VM fournie (`162.38.111.31`), joignable en SSH depuis le réseau de Polytech ou par VPN, avec un compte par membre, sans droits administrateur, et Docker en mode *rootless*. Un pipeline GitLab est imposé et la procédure de retour arrière est évaluée (critère E).

Critères de décision (fixés avant la comparaison)

(1) Retour arrière rapide et sûr ; (2) l'image déployée est exactement celle qui a été testée ; (3)

30. Codes HTTP : **409** : conflit avec l'état des données (capacité dépassée, facture déjà émise…).

charge limitée sur la VM ; (4) aucun secret dans le dépôt.

Options considérées

A. `git pull` puis `docker compose build` sur la VM : simple, mais compilation sur la VM (3), image non testée (2), retour arrière par recompilation (1).

B. Images construites en CI, poussées dans le registre GitLab avec le SHA court du commit comme étiquette ; la VM fait `pull` puis `up -d` avec `TAG=<sha>`.

C. Fichiers déployés sans Docker en production (`rsync` et gestionnaire de processus) : écart entre développement et production.

Décision

Option B. L'application tourne sous le Docker rootless du responsable du déploiement (`mathieu.marty`) ; la CI s'y connecte avec une clé SSH dédiée, stockée dans une variable GitLab protégée et masquée. Retour arrière : relancer le job `deploy` avec l'étiquette précédente. Migrations SQL appliquées par l'API au démarrage, uniquement additives.

Conséquences

Positives : retour arrière en moins d'une minute, traçabilité commit → image → version en ligne. *Négatives* : dépendance au registre GitLab (quota de stockage) ; un retour arrière applicatif ne défait pas une migration, d'où la règle des migrations additives (on ajoute d'abord, on supprime dans une version ultérieure) ; en Docker rootless, les ports inférieurs à 1024 ne sont pas publiables sans réglage de l'administrateur *[à confirmer : 80/443 ou ports hauts]*, le démon doit être maintenu après déconnexion (`loginctl enable-linger`), et seul le compte qui l'héberge peut gérer les conteneurs ; la VM doit disposer d'un jeton de lecture du registre.

ADR-011 - Table unique `structure` à indicateurs de rôle ; la réservation porte l'exposant

Statut : Acceptée **Date :** 24/10/2026 (proposée le 23/10/2026)

Contexte

Les données fournies décrivent des éditeurs avec deux indicateurs, `exposant` et `distributeur` (164 exposants, 3 cumulent les deux, 143 ni l'un ni l'autre). Le client introduit la notion d'*exposant* : l'association se réserve des espaces (ni jeux ni facture) ; un prestataire, qui n'est pas éditeur, loue un espace pour animer les jeux d'un ou plusieurs éditeurs (facturé, avec jeux). Le brouillon appelait « organisateur » la structure qui réserve et « animateur » une structure qui « distribue ».

Critères de décision (fixés avant la comparaison)

(1) Pas de doublon d'une même entreprise ni de ses contacts ; (2) couvrir tous les cas d'exposant du client ; (3) requêtes de suivi (UC6) simples ; (4) contraintes vérifiables.

Options considérées

A. Tables séparées `editeur`, `exposant`, `distributeur` : explicite, mais doublons d'entreprises et de contacts (1).

B. Héritage (`structure` et tables filles) : propre en théorie, mais jointures systématiques pour très peu d'attributs propres à chaque rôle.

C. Table unique `structure` avec `est_editeur`, `est_distributeur`, `est_exposant`, `est_association` ; `reservation.exposant_id` ; `reservation.facturable` ; les jeux d'une réservation peuvent appartenir à n'importe quel éditeur.

Décision

Option C, sous réserve de validation par le client. « Animateur » est renommé « distributeur », conformément aux données *[à confirmer : sens exact de « distributeur » et de `jeu.distributeur_id`]*. Une seule structure peut porter `est_association` (index unique partiel).

Conséquences

Positives : une entreprise, une fiche ; les trois cas d'exposant tiennent dans le même modèle.

Négatives : la cohérence entre jeux présentés et exposant (un prestataire anime-t-il seulement pour des éditeurs déclarés ?) n'est pas contrainte ; colonnes renommées par rapport au brouillon (`organisateur_id` → `exposant_id`).

ADR-012 - Stockage des PDF : règles en fichiers, factures regénérées

Statut : Proposée **Date :** 24/10/2026

Contexte

L'application manipule deux sortes de PDF. Les **règles des jeux** sont des fichiers déposés tels quels par le gestionnaire de jeux : l'application ne les lit pas, elle les garde et les renvoie. Les **factures** sont produites par l'application à partir de ses propres données.

Critères de décision (fixés avant la comparaison)

(1) Base de données légère, donc requêtes et sauvegardes rapides ; (2) une facture déjà envoyée doit pouvoir être reproduite à l'identique ; (3) simplicité ; (4) peu de données à sauvegarder à part.

Options considérées

A. Tout mettre dans la base (colonne binaire `bytea`) : une seule sauvegarde, mais la base grossit de plusieurs gigaoctets avec environ 1 500 jeux (critère 1).

B. Tout mettre en fichiers, factures comprises : simple à servir, mais les factures deviennent des fichiers à sauvegarder à part et ne sont plus interrogeables (critères 3 et 4).

C. Règles en fichiers, factures regénérées : les règles vont dans le volume `regles` et la base ne garde que leur nom (`jeu.regles_pdf`) ; une facture n'est pas stockée en PDF, elle est regénérée à chaque téléchargement depuis ses données figées en base (ADR-007).

Décision

Option C. Chaque sorte de PDF est rangée selon sa nature : un fichier reçu reste un fichier, un document que l'application fabrique est refait à partir des données.

Conséquences

Positives : base légère ; factures toujours reproductibles et interrogeables (liste, reste à payer).
Négatives : le volume `regles` doit être sauvegardé en plus de la base ; l'API doit supprimer le fichier quand une règle est remplacée ou supprimée ; si la mise en page des factures change, les anciennes factures changent d'aspect (pas de contenu) *[à confirmer : faut-il conserver le fichier exact envoyé à l'exposant ?]*.

ADR-013 - Niveau hiérarchique stocké dans le compte : `utilisateur.niveau` et `est_tresorier`

Statut : Acceptée **Date :** 24/10/2026

Contexte

L'ADR-001 fixe des rôles hiérarchiques à niveaux et un rôle trésorier parallèle. Le premier script SQL stockait les rôles d'un compte dans une table `utilisateur_role`, une ligne par rôle, et calculait le niveau comme le plus élevé de ces rôles. Or, avec des rôles hiérarchiques, posséder deux rôles hiérarchiques n'apporte rien : le plus élevé inclut l'autre. De plus, un compte qui n'avait que le rôle trésorier avait un niveau nul, et les routes de lecture le bloquaient, alors que le client veut qu'il voie tout comme un membre (Q21).

Critères de décision (fixés avant la comparaison)

(1) Fidélité à l'ADR-001 ; (2) rendre impossibles en base les états absurdes (compte sans niveau, trésorier sans accès en lecture) ; (3) simplicité des requêtes et du contrôle d'accès ; (4) facilité d'explication.

Options considérées

A. Table `utilisateur_role` (une ligne par rôle, niveau calculé) : souple, mais permet des combinaisons inutiles et le trésorier sans niveau (critère 2), et demande une jointure à chaque connexion (critère 3).

B. Colonne `niveau` (1 à 5) et indicateur `est_tresorier` dans `utilisateur` : un seul niveau par compte, obligatoire ; le trésorier s'ajoute par un booléen.

C. Table de référence des rôles (nom, niveau) et clé étrangère : noms des rôles en base, mais une table de plus pour six valeurs stables, déjà écartée par l'ADR-001 (option RBAC).

Décision

Option B. `niveau` est `NOT NULL` avec `CHECK (niveau BETWEEN 1 AND 5)` ; les noms des rôles ne figurent que dans le code. Un trésorier a donc toujours au moins le niveau 1 (membre), ce qui règle le blocage de la lecture.

Conséquences

Positives : une seule lecture à la connexion ; un compte a toujours un niveau valide ; `exigerNiveau(n)` compare un entier, `exigerRole('tresorier')` lit un booléen. *Négatives* : ajouter un rôle intermédiaire oblige à renuméroter les niveaux existants par une migration ; un futur rôle parallèle autre que trésorier demanderait une nouvelle colonne.

B Glossaire

Sigles et abréviations.

`ADR` *Architecture Decision Record*, fiche de décision d'architecture
`API` *Application Programming Interface*, interface de programmation
`C4` *Context, Containers, Components, Code*, modèle de diagrammes d'architecture
`CI/CD` intégration et déploiement continus
`CORS` *Cross-Origin Resource Sharing*, règles d'accès entre origines
`CRM` *Customer Relationship Management*, gestion de la relation client
`CSP` *Content Security Policy*, politique de sécurité du contenu
`CSRF` *Cross-Site Request Forgery*, requête falsifiée depuis un autre site
`DAT` Document d'Architecture Technique
`FK / PK` clé étrangère (*foreign key*) / clé primaire (*primary key*)
`HTTP(S)` protocole d'échange du Web (chiffré par TLS)
`JSON` *JavaScript Object Notation*, format d'échange de données
`JWT` *JSON Web Token*, jeton d'authentification signé
`LTS` *Long Term Support*, version à support long
`ORM` *Object-Relational Mapping*, outil de correspondance objet-relationnel (non utilisé)
`REST` *Representational State Transfer*, style d'API fondé sur des ressources
`SHA` empreinte qui identifie un commit Git
`SPA` *Single Page Application*, application monopage
`SQL` *Structured Query Language*, langage de requête relationnel
`SSH` *Secure Shell*, connexion à distance chiffrée
`TLS` *Transport Layer Security*, chiffrement des échanges
`TIMESTAMPTZ` *timestamp with time zone*, date et heure avec fuseau horaire (type PostgreSQL)
`UML` *Unified Modeling Language*, notation de modélisation
`VM` *Virtual Machine*, machine virtuelle
`VPN` *Virtual Private Network*, réseau privé virtuel
`XSS` *Cross-Site Scripting*, injection de script dans une page

Termes du domaine.

Structure	Entreprise ou association connue de l'application ; peut être éditeur, distributeur et exposant à la fois.
Éditeur	Structure qui publie des jeux.
Distributeur	Structure qui distribue des jeux (indicateur `distributeur` des données fournies) *[à confirmer : rôle exact].*
Exposant	Structure qui réserve un espace sur un festival : éditeur, prestataire qui anime pour des éditeurs, ou l'association elle-même (réservation non fac-

	turée).
Réservation	Dossier d'un exposant pour un festival : workflow de suivi, tables par zone, remises, repas, jeux, factures.
Zone tarifaire	Partie d'un festival avec un prix de table, une capacité et une remise de précommande.
Zone du plan	Emplacement physique rattaché à une zone tarifaire, où sont placés les jeux.
Contact de suivi	Prise de contact datée avec un exposant pour un festival, avec notes.
Rôles	Administrateur, commercial (le client dit aussi « super organisateur »), gestionnaire de plan, gestionnaire de jeux, membre de l'organisation, trésorier (section 1 et ADR-001).
Visiteur	Personne qui consulte la liste des festivals et leurs détails, sans compte.

C Matrice des droits

É = lecture et édition ; L = lecture ; – = aucun accès. Chaque compte a un niveau de 1 à 5 ; le trésorier est un indicateur qui s'ajoute à ce niveau (ADR-013).

Tableau 7 – Matrice des droits par rôle

Donnée ou action	Adm.	Com.	Plan	Jeux	Memb.	Trés.
Comptes et attribution des rôles	É	–	–	–	–	–
Festivals, zones tarifaires et zones du plan	É	L	L	L	L	L
Structures (éditeurs, distributeurs, exposants)	É	É	L	L	L	L
Contacts, contacts de suivi, notes	É	É	–	–	–	L
Réservation : tables, remises, repas, workflow, validation	É	É	–	–	–	L
Réservation validée : tables par zone, jeux, plan	É	É	L	L	L	L
Ajout des jeux, affectation aux tables et au plan	É	É	É	L	L	L
Fiches des jeux	É	É	É	É	L	L
Factures et paiements	L	L	–	–	–	É

L'administrateur n'a que la lecture des factures : seul le trésorier les crée et les modifie (ADR-003) *[à confirmer : un administrateur peut-il agir en trésorier ? sinon lui attribuer aussi ce rôle].*

D Les niveaux du modèle C4

Le modèle C4 (Context, Containers, Components, Code) décrit un système par zooms successifs. Le DAT n'utilise que les deux premiers niveaux ; le niveau 3 est couvert par le texte de la section 4 et le niveau 4 par le dépôt.

Tableau 8 – Les quatre niveaux du modèle C4 et leur usage dans ce DAT

Niveau	Ce qu'il montre	Public visé	Dans ce DAT
1. Contexte	Le système vu de l'extérieur : une seule boîte, ses utilisateurs et les systèmes externes avec lesquels il interagit ; ni technologie ni protocole.	Tout le monde, y compris les non-techniciens.	Oui : figure 1 (p. 4), section 3.1.
2. Briques logicielles (*containers*)	L'intérieur du système : les applications et les stockages de données, leurs technologies et la façon dont ils communiquent ; aucun élément de déploiement.	Architectes, développeurs, exploitation.	Oui : figure 2 (p. 5), section 3.2 ; déploiement à part, figure 5 (p. 11).

Tableau 8 – Les quatre niveaux du modèle C4 (suite)

Niveau	Ce qu'il montre	Public visé	Dans ce DAT
3. Composants	L'intérieur d'une brique : ses modules principaux et leurs responsabilités.	Architectes et développeurs.	Non dessiné : couches décrites en section 4, p. 5.
4. Code	Classes, fonctions, tables.	Développeurs.	Non : laissé au dépôt ; schéma de données en figure 3 (p. 8).

E Besoins, règles de gestion et cas d'utilisation

Besoins fonctionnels

BF1 Créer, modifier et consulter des festivals (nom unique, lieu obligatoire) ; copier d'un festival précédent les zones tarifaires, le nombre de tables, la taille de table et la liste des exposants (sans les notes) ; sélectionner le festival courant (par défaut le dernier créé).

BF2 Définir les zones tarifaires d'un festival (au moins une) : capacité, prix de table, remise de précommande, tables restantes, indicateur « complet ». L'unité est la table ; une surface en m^2 est convertie en tables.

BF3 Définir les zones du plan (au moins une), chacune rattachée à une zone tarifaire.

BF4 Gérer les structures et leurs rôles (éditeur, distributeur, exposant), avec tri et recherche.

BF5 Gérer les contacts d'une structure, les contacts de suivi datés et les notes propres à chaque festival.

BF6 Gérer les jeux : nom, auteurs, éditeur, type, âge minimum et maximum, nombre de joueurs, règles (PDF) ; tri selon critères.

BF7 Gérer une réservation : tables par zone, remises, repas, jeux (nombre d'exemplaires, part de table), déplacement de l'exposant, animation par l'association, réservation non facturée pour l'association.

BF8 Calculer le prix, appliquer les remises, convertir les m^2 et calculer les tables et l'espace restants.

BF9 Suivre les exposants du festival courant : statut du workflow, date du dernier contact, case « contacté » directement dans la liste, tables et prix si réservé, tri par statut.

BF10 Créer les factures PDF d'une réservation, les annuler sans les supprimer, les rembourser, enregistrer les paiements et calculer le reste à payer.

BF11 Réserver ces actions de facturation au trésorier ; une facture créée n'est plus modifiée.

BF12 Permettre au visiteur de consulter la liste des festivals (cartes : nom, tables disponibles au total et par zone) et leurs détails.

BF13 Réserver des repas pour les exposants qui se déplacent.

BF14 Authentifier par login et mot de passe ; rôles hiérarchiques et rôle trésorier parallèle.

BF15 Passer automatiquement à « considéré absent », à la date du festival, les réservations restées sans réponse.

BF16 Valider une réservation, ce qui la rend visible, dans sa forme finale, aux rôles inférieurs.

BF17 Masquer les informations commerciales aux rôles qui n'en ont pas besoin.

BF18 Suivre la liste des jeux : demandée, obtenue, jeux reçus (dates).

BF19 Lister les factures du festival courant avec leur état.

BF20 Lister les jeux du festival avec leur zone du plan, triables par nom, auteur, éditeur, zone.

Règles de gestion

Conversion m^2 / tables

Nombre de tables = surface demandée / `taille_table_m2` (paramétrable par festival, 4 m^2 par

défaut, Q23), arrondi à l'unité supérieure.

Limite de commande
Refus si la réservation dépasse la capacité de la zone ou l'espace restant du festival ; somme des capacités des zones = surface totale / taille de table (Q25) *[à confirmer : interprétation de Q25]*.

Prix
Somme sur les zones de (tables payantes × prix de la table), au prix remisé si la réservation est confirmée avant la date limite de la zone ; plus les repas.

Remises
Trois types : précommande, tables offertes, montant sur le total. Le client a répondu « pas de cumul » (Q19) *[à confirmer : la remise de précommande est-elle aussi exclusive des deux autres ?]*.

Tables restantes
Capacité moins tables des réservations actives ; les réservations annulées ou absentes libèrent leurs tables.

Placement
Un jeu est placé dans une zone du plan liée à une zone tarifaire de sa réservation ; la somme des parts de table ne dépasse ni les tables réservées dans cette zone tarifaire, ni les tables de la zone du plan.

Copie d'un festival
À la création, l'administrateur peut choisir un festival source (`copieDe`). Sont recopiés : la surface, la taille de table, les zones tarifaires (nom, capacité, prix, remise ; la date limite de remise est à ressaisir) et les zones du plan. Pour chaque exposant du festival source, un nouveau dossier de réservation est créé au statut `pas_contacte`. Ne sont pas recopiés : les notes, les contacts de suivi, les tables réservées, les remises accordées, les jeux placés et les factures (Q10). Le tout se fait dans une seule transaction.

Suppression d'une zone
Interdite si une réservation la référence.

Validation
Voir ADR-002.

Factures et paiements
Voir ADR-003 et ADR-007. Reste à payer = somme des factures non annulées − somme des paiements encaissés. Chèque refusé : paiement marqué refusé, réservation annulée, structure marquée cliente douteuse.

Annulation
Sans facture : annulation simple. Avec facture : facture annulée ; si elle était payée, « à rembourser » puis « remboursée ».

Workflow
Statuts `pas_contacte`, `contacte`, `discussion`, `sera_absent`, `considere_absent`, `present`, `annulee` ; « facturé » et « payé » des notes initiales sont dérivés des factures *[à confirmer : liste exacte]*. Changements manuels, sauf le passage automatique (ADR-009).

Cas d'utilisation

UC1 Créer, modifier ou copier un festival (administrateur).
UC2 Consulter les festivals (tous, y compris visiteur).
UC3 Gérer les structures (commercial).
UC4 Gérer les contacts et contacts de suivi (commercial).
UC5 Gérer les jeux (gestionnaire de jeux).
UC6 Suivre les exposants du festival courant (commercial).
UC7 Gérer une réservation : négociation et validation (commercial), puis jeux et plan (gestionnaire de plan).
UC8 Facturer et enregistrer les paiements (trésorier).

UC9 Gérer les comptes et les rôles (administrateur).
UC10 Consulter les jeux du festival et le plan (membre de l'organisation).

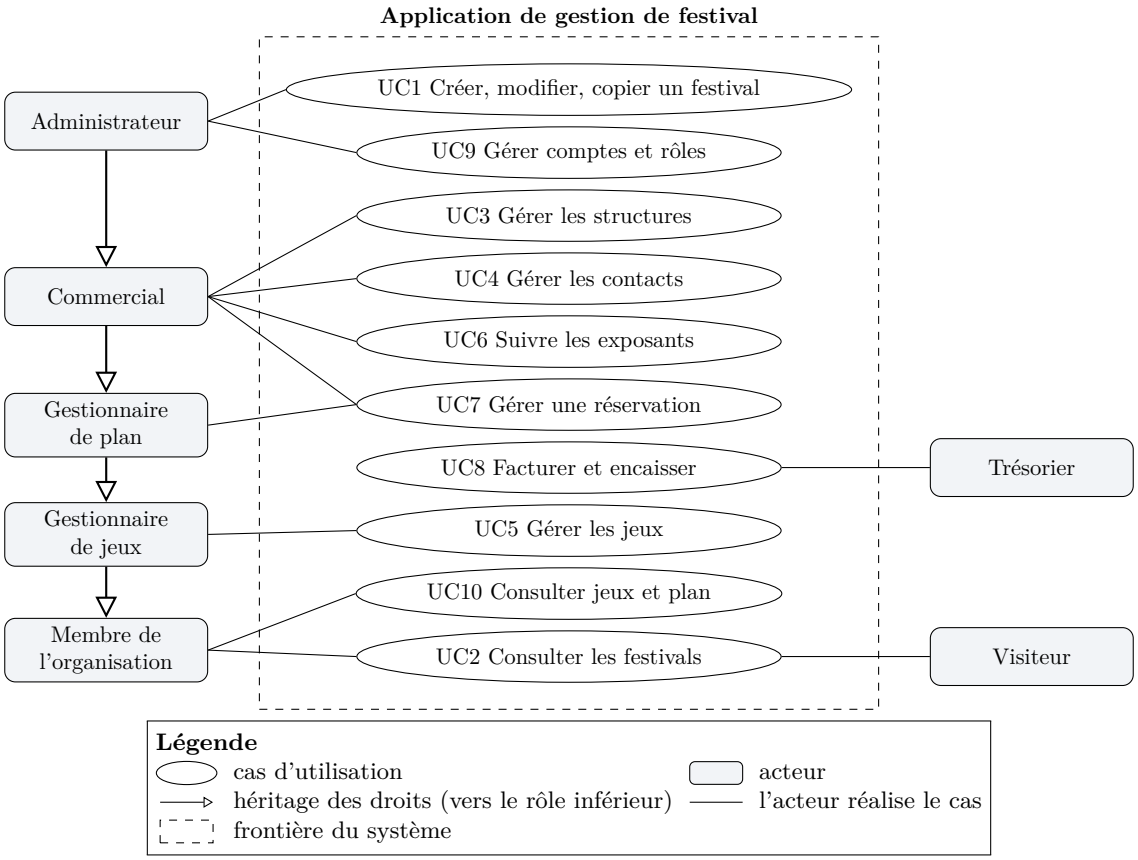


FIGURE 7 – [Diagramme de cas d'utilisation, UML] Le trésorier, hors hiérarchie, voit aussi ce que voit un membre.

F Comptes rendus des entrevues

Questions posées au client et ses réponses, reportées dans le document de besoin (`docs/besoin.tex`). Les questions 1 à 19 datent de l'entrevue du 07/10/2026, les questions 20 à 26 de l'entrevue du 23/10/2026.

TABLEAU 9 – Questions posées au client et réponses (entrevues du 07/10 et du 23/10/2026)

N°	Question	Réponse du client
Entrevue du 07/10/2026		
1	Formule de conversion des m^2 en tables ?	L'unité est la table ; conversion selon la superficie d'une table, dans la limite de l'espace restant.
2	Prix par m^2 : division par 4 ou 4,5 ?	Surface de 4 000 m^2 ; la taille de table est traitée en Q23.
3	Limite de m^2 ou de tables commandables ?	La limite est l'espace.
4	Nombre de personnes par table ?	Sans intérêt.
5	Remise en tables et en argent sur une même réservation ?	Trois types : montant sur le total, prix de table réduit avant une date, table offerte.
6	Traitement d'une annulation ?	Facture non payée : annulation simple. Facture éditée : annulée, à rembourser.

Tableau 9 – Questions posées au client et réponses (suite)

N°	Question	Réponse du client (suite)
7	Paiement en plusieurs fois, plusieurs factures ?	Repas des exposants qui se déplacent ; ajout de tables : nouvelle facture.
8	Garder date et moyen de paiement ?	Date d'émission et de paiement ; virement ou chèque ; chèque refusé : annulation.
9	Informations d'un contact ?	Nom, prénom, portable, fixe, fonction, email. Client douteux si chèque refusé.
10	Données copiées d'un festival précédent ?	CRM et réservation ; notes propres au festival non copiées ; zones, tables, conversion, exposants et éditeurs réutilisés.
11	« Considéré absent » : manuel ou automatique ?	Statuts manuels, sauf factures ; absent automatiquement à la date du festival.
12	Connexion sécurisée ?	Oui. Rôles précisés ensuite (ADR-001).
13	Nombres à virgule pour les tables d'un jeu ?	Une réservation porte des tables et une liste de jeux avec leur occupation.
14	Critère de formation des jeux le jour J ?	Déplacement de l'exposant et choix d'animer ou non.
15	Accès séparé pour les animateurs de l'association ?	Pas de réponse directe ; déduit de Q18 : aucun compte.
16	Lieu géographique du festival ?	Oui.
17	Délai d'absence, zone en attente, partage d'une table ?	Aucune compensation ; deux jeux d'une même réservation peuvent partager une table.
18	Staff de présentation des jeux ?	Aucune gestion des bénévoles (environ 300).
19	Cumul de remises ?	Pas possible.
Entrevue du 23/10/2026		
20	Comment le commercial signale-t-il un ajout au trésorier ?	En interne, hors application, mais l'application doit tout dater.
21	Le trésorier peut-il lire les fiches de jeux ?	Oui, il voit tout comme les autres membres.
22	Retards de paiement ?	Autorisés, avec une notion d'impayé.
23	Taille d'une table fixée ?	Paramétrable par festival, 4 m^2 par défaut.
24	Le commercial voit-il seulement ses réservations ?	Non, toutes.
25	Les zones peuvent-elles faire perdre des m^2 ?	Non.
26	Le visiteur consulte-t-il les festivals sans compte ?	Oui, sans compte.

G Liste complète des endpoints

Préfixe `/api/v1`. Rôle minimal entre parenthèses : un rôle inclut les rôles inférieurs (« commercial » vaut aussi pour l'administrateur) ; le trésorier, hors hiérarchie, est cité à part. Une suppression refusée parce que des données y font référence renvoie 409. La dernière colonne donne les codes de réponse possibles [31].

31. Codes HTTP : **200** : réussite ; **201** : ressource créée ; **204** : réussite, sans contenu à renvoyer ; **400** : requête invalide (données mal formées) ; **401** : non authentifié (pas de jeton ou jeton expiré) ; **403** : authentifié mais droits insuffisants ; **404** : ressource introuvable ; **409** : conflit avec l'état des données (capacité dépassée, facture déjà émise…) ; **413** : fichier trop volumineux ; **429** : trop de requêtes (limitation de débit) ; **503** : service indisponible (base injoignable, par exemple).

Tableau 10 – Liste complète des endpoints de l'API

Méthode	Chemin	**Description**	**Codes**
Authentification			
POST	/auth/connexion	Connexion, pose le cookie (public)	200, 400, 401, 429
POST	/auth/deconnexion	Déconnexion, efface le cookie	204
GET	/auth/moi	Profil du compte connecté (login, niveau, trésorier)	200, 401
PATCH	/auth/moi/mot-de-passe	Changement de son propre mot de passe (tout compte connecté) *[à confirmer : le membre peut-il modifier lui-même son compte ?]*	204, 400, 401
Comptes (ADR-013)			
GET, POST	/utilisateurs	Liste ; création d'un compte avec son niveau et l'indicateur trésorier (admin)	200, 201, 400, 401, 403, 409
GET, PUT	/utilisateurs/:id	Détail ; modification du niveau ou de l'indicateur ; le dernier administrateur ne peut pas être rétrogradé (admin)	200, 400, 401, 403, 404, 409
PATCH	/utilisateurs/:id/desactivation	Désactivation d'un compte, qui garde son historique (admin)	200, 401, 403, 404, 409
DELETE	/utilisateurs/:id	Suppression d'un compte qui n'a encore aucune action enregistrée, sinon 409 (admin)	204, 401, 403, 404, 409
Festivals et zones			
GET	/festivals	Liste des festivals, cartes avec tables disponibles (public)	200
POST	/festivals	Création ; avec `copieDe`, recopie un festival précédent selon la règle « Copie d'un festival » de l'annexe E (p. 23) (admin)	201, 400, 401, 403, 404, 409
GET	/festivals/:id	Détail et disponibilités (public)	200, 404
PUT, DELETE	/festivals/:id	Modification ; suppression refusée (409) si des réservations existent (admin)	200, 204, 400, 401, 403, 404, 409
GET	/festivals/:id/zones-tarifaires	Forme publique (nom, capacité, tables restantes) pour le visiteur ; prix et remise dès le niveau membre	200, 404
POST	/festivals/:id/zones-tarifaires	Création d'une zone tarifaire (admin)	201, 400, 401, 403, 404
PUT, DELETE	/festivals/:id/zones-tarifaires/:zoneId	Modification ; suppression refusée (409) si la zone est réservée (admin)	200, 204, 400, 401, 403, 404, 409
GET	/festivals/:id/zones-plan	Zones du plan (public)	200, 404
POST	/festivals/:id/zones-plan	Création d'une zone du plan (admin)	201, 400, 401, 403, 404
PUT, DELETE	/festivals/:id/zones-plan/:zoneId	Modification ; suppression refusée (409) si des jeux y sont placés (admin)	200, 204, 400, 401, 403, 404, 409
GET	/festivals/:id/suivi	Tableau de suivi des exposants du festival (commercial)	200, 401, 403, 404
GET	/festivals/:id/jeux	Jeux du festival avec leur zone du plan, triables (membre)	200, 401, 403, 404
GET	/festivals/:id/factures	Factures du festival et leur état (commercial, trésorier)	200, 401, 403, 404
Structures, contacts et jeux			
GET, POST	/structures	Liste filtrée par rôle (membre) ; création (commercial)	200, 201, 400, 401, 403
GET, PUT, DELETE	/structures/:id	Détail (membre) ; modification ; suppression refusée (409) si la structure a des réservations ou des jeux (commercial)	200, 204, 400, 401, 403, 404, 409
GET, POST	/structures/:id/contacts	Contacts de la structure (lecture commercial et trésorier) ; ajout (commercial)	200, 201, 400, 401, 403, 404
PUT, DELETE	/structures/:id/contacts/:contactId	Modification, suppression d'un contact (commercial)	200, 204, 400, 401, 403, 404
GET, POST	/jeux	Liste triable, filtrable par éditeur (`?editeur=…`) (membre) ; création (gestionnaire de jeux)	200, 201, 400, 401, 403
GET, PUT, DELETE	/jeux/:id	Détail (membre) ; modification ; suppression refusée (409) si le jeu est présenté sur un festival (gestionnaire de jeux)	200, 204, 400, 401, 403, 404, 409
GET, PUT	/jeux/:id/regles	Téléchargement du PDF des règles (membre) ; dépôt ou remplacement, taille et type contrôlés (gestionnaire de jeux) (ADR-012)	200, 400, 401, 403, 404, 413
GET	/types-jeu	Types de jeu (membre)	200, 401

Tableau 10 – Liste complète des endpoints de l'API (suite)

Méthode	Chemin	**Description**	**Codes**
Réservations			
GET, POST	/reservations	Liste filtrée par festival : forme complète au niveau 4 ou pour le trésorier, forme publique des seules réservations validées sinon (ADR-002) ; création (commercial)	200, 201, 400, 401, 403, 409
GET, PUT	/reservations/:id	Détail (mêmes formes) ; modification des tables, remises et notes (commercial)	200, 400, 401, 403, 404, 409
PATCH	/reservations/:id/statut	Changement de statut du workflow (commercial)	200, 400, 401, 403, 404
PATCH	/reservations/:id/validation	Validation (commercial)	200, 401, 403, 404, 409
PATCH	/reservations/:id/annulation	Annulation, sans suppression ; les factures émises sont annulées ou passent « à rembourser » (commercial)	200, 401, 403, 404, 409
GET, POST	/reservations/:id/contacts-suivi	Historique des contacts de suivi (commercial, trésorier) ; ajout d'un contact daté (commercial)	200, 201, 400, 401, 403, 404
GET, PUT	/reservations/:id/jeux	Jeux, parts de table et placement, seulement après validation (lecture membre, écriture gestionnaire de plan)	200, 400, 401, 403, 404, 409
PUT	/reservations/:id/repas	Repas des exposants (commercial)	200, 400, 401, 403, 404
Facturation (ADR-003, ADR-007)			
GET, POST	/reservations/:id/factures	Factures de la réservation (lecture commercial et trésorier) ; création (trésorier)	200, 201, 401, 403, 404, 409
GET	/factures/:id/pdf	PDF regénéré depuis la facture figée (commercial, trésorier)	200, 401, 403, 404
PATCH	/factures/:id/annulation	Annulation ; passe « à rembourser » si elle était payée (trésorier)	200, 401, 403, 404, 409
PATCH	/factures/:id/remboursement	Remboursement effectué (trésorier)	200, 401, 403, 404, 409
GET, POST	/factures/:id/paiements	Paiements (lecture commercial et trésorier) ; enregistrement (trésorier)	200, 201, 400, 401, 403, 404, 409
PATCH	/paiements/:id/refus	Chèque refusé : paiement marqué refusé, réservation annulée, structure marquée cliente douteuse, dans une même transaction (trésorier)	200, 401, 403, 404, 409
Exploitation			
GET	/sante	Test de santé utilisé par le déploiement (public) ; 503 si la base ne répond pas	200, 503

H Rôle des tables du modèle de données

Le tableau 11 (p. 28) explique les tables dont le nom ou le rôle peut prêter à confusion.

Tableau 11 – Rôle des tables qui peuvent prêter à confusion

Table	**Ce qu'elle représente**
structure	Une entreprise ou une association, quel que soit son rôle. Les indicateurs est_editeur, est_distributeur, est_exposant, est_association disent ce qu'elle est ; une même structure peut en cumuler plusieurs (ADR-011).
contact	Une *personne* chez une structure (nom, téléphones, fonction). Elle existe d'un festival à l'autre.
contact_suivi	Un *événement* : une prise de contact datée avec un exposant, pour un festival donné, avec ses notes et le compte qui l'a faite. C'est l'historique CRM, qui fournit la date du dernier contact.
reservation	Le *dossier* d'un exposant pour un festival, créé dès le premier contact, même s'il ne réserve jamais rien. Il porte le statut du workflow, les remises, les repas, les notes et la validation. Une seule réservation par exposant et par festival.
reservation_zone	Les tables réservées par zone tarifaire : une ligne par couple (réservation, zone), d'où sa clé primaire composée. C'est elle qui sert au calcul du prix et des tables restantes.

Tableau 11 – Rôle des tables qui peuvent prêter à confusion (suite)

Table	Ce qu'elle représente
`reservation_jeu`	Les jeux présentés par une réservation, avec le nombre d'exemplaires, la part de table occupée (0,5 si deux jeux partagent une table) et la zone du plan où ils sont placés.
`zone_tarifaire` / `zone_plan`	La zone tarifaire fixe le *prix* et la capacité ; la zone du plan est un *emplacement* physique, rattaché à une zone tarifaire, où l'on place les jeux.
`facture` / `facture_ligne`	La facture porte le numéro, le total, le statut et les coordonnées figées de l'exposant. Ses lignes sont copiées de la réservation au moment de l'émission et ne changent plus : chaque facture garde ses propres lignes, et une nouvelle facture reçoit les siennes (ADR-007).
`paiement`	Un règlement (virement ou chèque) d'une facture ; une facture peut en avoir plusieurs. Un chèque refusé est marqué `refuse`, sans suppression.
`utilisateur`	Un compte de l'association (pas un exposant). `niveau` vaut 1 (membre) à 5 (administrateur) ; `est_tresorier` ajoute le rôle trésorier (ADR-013). Les autres tables y renvoient pour dater et attribuer chaque action (Q20).

I Script SQL initial

```sql
-- db/migrations/001_initial.sql (commentaires sans accents : listings)

CREATE TABLE festival (
  id SERIAL PRIMARY KEY,
  nom VARCHAR(120) NOT NULL UNIQUE,
  lieu VARCHAR(120) NOT NULL,
  date_debut DATE NOT NULL,
  date_fin DATE NOT NULL CHECK (date_fin >= date_debut),
  surface_totale_m2 INT NOT NULL CHECK (surface_totale_m2 > 0),
  taille_table_m2 NUMERIC(4,2) NOT NULL DEFAULT 4.0 CHECK (taille_table_m2 > 0),
  prix_repas NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (prix_repas >= 0), -- a confirmer
  cree_le TIMESTAMPTZ NOT NULL DEFAULT now() -- festival courant par defaut
);

CREATE TABLE zone_tarifaire (
  id SERIAL PRIMARY KEY,
  festival_id INT NOT NULL REFERENCES festival(id) ON DELETE CASCADE,
  nom VARCHAR(40) NOT NULL,
  capacite INT NOT NULL CHECK (capacite > 0), -- en tables
  prix_table NUMERIC(10,2) NOT NULL CHECK (prix_table >= 0),
  remise_precommande_pct NUMERIC(5,2) NOT NULL DEFAULT 0
                         CHECK (remise_precommande_pct BETWEEN 0 AND 100),
  date_limite_remise DATE,
  UNIQUE (festival_id, nom)
);
-- Suppression d'une zone reservee : bloquee par la FK de reservation_zone.

CREATE TABLE zone_plan (
  id SERIAL PRIMARY KEY,
  festival_id INT NOT NULL REFERENCES festival(id) ON DELETE CASCADE,
  zone_tarifaire_id INT NOT NULL REFERENCES zone_tarifaire(id),
  nom VARCHAR(80) NOT NULL,
  nb_tables INT NOT NULL CHECK (nb_tables > 0),
  UNIQUE (festival_id, nom)
);

CREATE TABLE structure (
  id SERIAL PRIMARY KEY,
  nom VARCHAR(120) NOT NULL,
  email VARCHAR(160),
  telephone VARCHAR(30),
  adresse VARCHAR(255), -- utile sur la facture (a confirmer)
  pays VARCHAR(60),
  ville VARCHAR(80),
  logo_url VARCHAR(255), -- present dans les donnees fournies
```

```
  est_editeur BOOLEAN NOT NULL DEFAULT FALSE,
  est_distributeur BOOLEAN NOT NULL DEFAULT FALSE,
  est_exposant BOOLEAN NOT NULL DEFAULT FALSE,
  est_association BOOLEAN NOT NULL DEFAULT FALSE,
  client_douteux BOOLEAN NOT NULL DEFAULT FALSE -- cheque refuse
);
CREATE UNIQUE INDEX une_seule_association ON structure (est_association) WHERE est_association;
CREATE INDEX structure_nom_idx ON structure (nom);

CREATE TABLE contact (
  id SERIAL PRIMARY KEY,
  structure_id INT NOT NULL REFERENCES structure(id) ON DELETE CASCADE,
  nom VARCHAR(80) NOT NULL,
  prenom VARCHAR(80),
  tel_mobile VARCHAR(30),
  tel_fixe VARCHAR(30),
  fonction VARCHAR(80),
  email VARCHAR(160)
);
CREATE INDEX contact_structure_idx ON contact (structure_id);

-- Comptes de l'association (pas les exposants). ADR-013 : le role
-- hierarchique est un niveau entier, le tresorier un indicateur a part.
-- Niveaux : 1 membre_orga, 2 gestionnaire_jeux, 3 gestionnaire_plan,
-- 4 commercial, 5 admin. Les noms ne sont que dans le code.
CREATE TABLE utilisateur (
  id SERIAL PRIMARY KEY,
  login VARCHAR(60) NOT NULL UNIQUE,
  mot_de_passe_hash VARCHAR(255) NOT NULL,
  niveau SMALLINT NOT NULL CHECK (niveau BETWEEN 1 AND 5),
  est_tresorier BOOLEAN NOT NULL DEFAULT FALSE,
  actif BOOLEAN NOT NULL DEFAULT TRUE,
  cree_le TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Un tresorier a au moins le niveau 1 : il voit tout comme un membre (Q21).
-- Le dernier administrateur (niveau 5) ne peut etre ni supprime ni retrograde :
-- controle dans le service des comptes.

CREATE TABLE type_jeu (
  id SERIAL PRIMARY KEY,
  libelle VARCHAR(60) NOT NULL UNIQUE -- Tout public, Ambiance, Experts...
);

CREATE TABLE jeu (
  id SERIAL PRIMARY KEY,
  nom VARCHAR(160) NOT NULL,
  auteurs VARCHAR(255), -- descriptif uniquement
  editeur_id INT NOT NULL REFERENCES structure(id),
  distributeur_id INT REFERENCES structure(id),
  type_jeu_id INT REFERENCES type_jeu(id),
  age_min INT CHECK (age_min >= 0),
  age_max INT CHECK (age_max IS NULL OR age_max >= age_min),
  nb_joueurs_min INT CHECK (nb_joueurs_min > 0),
  nb_joueurs_max INT CHECK (nb_joueurs_max IS NULL OR nb_joueurs_max >= nb_joueurs_min),
  duree_min INT,
  prototype BOOLEAN NOT NULL DEFAULT FALSE,
  regles_pdf VARCHAR(255)
);
CREATE INDEX jeu_editeur_idx ON jeu (editeur_id);
CREATE INDEX jeu_nom_idx ON jeu (nom);

CREATE TABLE reservation (
  id SERIAL PRIMARY KEY,
  festival_id INT NOT NULL REFERENCES festival(id),
  exposant_id INT NOT NULL REFERENCES structure(id),
  createur_id INT NOT NULL REFERENCES utilisateur(id),
  statut VARCHAR(20) NOT NULL DEFAULT 'pas_contacte'
    CHECK (statut IN ('pas_contacte','contacte','discussion','sera_absent',
                      'considere_absent','present','annulee')),
  facturable BOOLEAN NOT NULL DEFAULT TRUE, -- FALSE : association
  reservee_le DATE, -- sert a la remise de precommande
  tables_offertes INT NOT NULL DEFAULT 0 CHECK (tables_offertes >= 0),
  remise_montant NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (remise_montant >= 0),
  nb_repas INT NOT NULL DEFAULT 0 CHECK (nb_repas >= 0),
  exposant_se_deplace BOOLEAN NOT NULL DEFAULT FALSE,
```

```
  animation_par_association BOOLEAN NOT NULL DEFAULT FALSE,
  liste_jeux_demandee_le DATE,
  liste_jeux_obtenue_le DATE,
  jeux_recus_le DATE,
  notes TEXT, -- notes propres au festival (CRM)
  validee_le TIMESTAMPTZ, -- NULL tant que non validee (ADR-002)
  cree_le TIMESTAMPTZ NOT NULL DEFAULT now(),
  modifiee_le TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (festival_id, exposant_id),
  -- Q19 : pas de cumul des remises (a confirmer pour la precommande)
  CHECK (tables_offertes = 0 OR remise_montant = 0)
);
CREATE INDEX reservation_suivi_idx ON reservation (festival_id, statut);
CREATE INDEX reservation_exposant_idx ON reservation (exposant_id);

CREATE TABLE reservation_zone (
  reservation_id INT NOT NULL REFERENCES reservation(id) ON DELETE CASCADE,
  zone_tarifaire_id INT NOT NULL REFERENCES zone_tarifaire(id) ON DELETE RESTRICT,
  nb_tables INT NOT NULL CHECK (nb_tables > 0),
  surface_m2 NUMERIC(8,2), -- si demande en m2 (trace)
  PRIMARY KEY (reservation_id, zone_tarifaire_id)
);
CREATE INDEX reservation_zone_zone_idx ON reservation_zone (zone_tarifaire_id);

CREATE TABLE reservation_jeu (
  id SERIAL PRIMARY KEY,
  reservation_id INT NOT NULL REFERENCES reservation(id) ON DELETE CASCADE,
  jeu_id INT NOT NULL REFERENCES jeu(id),
  zone_plan_id INT REFERENCES zone_plan(id), -- NULL tant que non place
  nb_exemplaires INT NOT NULL DEFAULT 1 CHECK (nb_exemplaires > 0),
  part_table NUMERIC(4,2) NOT NULL DEFAULT 1 CHECK (part_table > 0),
  UNIQUE (reservation_id, jeu_id)
);
-- Coherence zone_plan / zones de la reservation : controlee dans le service.

CREATE TABLE contact_suivi (
  id SERIAL PRIMARY KEY,
  reservation_id INT NOT NULL REFERENCES reservation(id) ON DELETE CASCADE,
  contact_id INT REFERENCES contact(id) ON DELETE SET NULL,
  auteur_id INT NOT NULL REFERENCES utilisateur(id),
  date_contact DATE NOT NULL,
  notes TEXT
);

CREATE TABLE facture (
  id SERIAL PRIMARY KEY,
  reservation_id INT NOT NULL REFERENCES reservation(id) ON DELETE RESTRICT,
  remplace_id INT REFERENCES facture(id), -- facture annulee remplacee
  numero VARCHAR(30) NOT NULL UNIQUE,
  exposant_nom VARCHAR(120) NOT NULL, -- coordonnees figees
  exposant_adresse VARCHAR(255),
  montant_total NUMERIC(10,2) NOT NULL CHECK (montant_total >= 0),
  creee_par INT NOT NULL REFERENCES utilisateur(id),
  emise_le TIMESTAMPTZ NOT NULL DEFAULT now(),
  statut VARCHAR(15) NOT NULL DEFAULT 'emise'
    CHECK (statut IN ('emise','annulee','a_rembourser','remboursee')),
  annulee_le TIMESTAMPTZ,
  remboursee_le TIMESTAMPTZ
);
CREATE INDEX facture_reservation_idx ON facture (reservation_id);

CREATE TABLE facture_ligne (
  id SERIAL PRIMARY KEY,
  facture_id INT NOT NULL REFERENCES facture(id) ON DELETE RESTRICT,
  libelle VARCHAR(200) NOT NULL, -- "Zone premium - 3 tables"
  quantite NUMERIC(8,2) NOT NULL,
  prix_unitaire NUMERIC(10,2) NOT NULL, -- negatif pour une remise
  montant NUMERIC(10,2) NOT NULL
);

CREATE TABLE paiement (
  id SERIAL PRIMARY KEY,
  facture_id INT NOT NULL REFERENCES facture(id) ON DELETE RESTRICT,
  montant NUMERIC(10,2) NOT NULL CHECK (montant > 0),
  moyen VARCHAR(10) NOT NULL CHECK (moyen IN ('virement','cheque')),
```

```sql
  date_emission DATE NOT NULL,
  date_paiement DATE,
  statut VARCHAR(10) NOT NULL DEFAULT 'encaisse'
    CHECK (statut IN ('encaisse','refuse')),
  enregistre_par INT NOT NULL REFERENCES utilisateur(id),
  enregistre_le TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX paiement_facture_idx ON paiement (facture_id);

-- ADR-007 : facture immuable (seuls le statut et ses dates changent)
CREATE FUNCTION facture_immuable() RETURNS trigger AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
    RAISE EXCEPTION 'Une facture ne se supprime pas, elle s''annule';
  END IF;
  IF (NEW.reservation_id, NEW.numero, NEW.montant_total, NEW.emise_le,
      NEW.creee_par, NEW.exposant_nom)
     IS DISTINCT FROM
     (OLD.reservation_id, OLD.numero, OLD.montant_total, OLD.emise_le,
      OLD.creee_par, OLD.exposant_nom) THEN
    RAISE EXCEPTION 'Une facture emise ne se modifie pas';
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER facture_immuable BEFORE UPDATE OR DELETE ON facture
  FOR EACH ROW EXECUTE FUNCTION facture_immuable();
-- Meme principe (non detaille ici) : facture_ligne sans UPDATE ni DELETE.

-- ADR-009 : statut effectif
CREATE VIEW v_reservation_statut AS
SELECT r.*,
  CASE WHEN f.date_debut <= CURRENT_DATE
        AND r.statut IN ('pas_contacte','contacte','discussion')
       THEN 'considere_absent' ELSE r.statut END AS statut_effectif
FROM reservation r JOIN festival f ON f.id = r.festival_id;
```

J Journal des évolutions

Tableau 12 – Journal des évolutions du DAT

Version	Date	Évolution	Raison	Type
`dat-v1` (brouillon)	09/10/2026	Version initiale	Cadrage avant le code	–
`dat-v1`	15/10/2026	ADR-001 à ADR-003 ; correction des cardinalités réservation–facture et facture–paiement du schéma (ex-ADR-004)	Précisions du client sur les rôles ; relecture	décidée ; correction
`dat-v1.1`	23/10/2026	Restructuration selon la trame (C4, sécurité, déploiement, CI/CD) ; ADR au format critères et options ; ADR-005 à ADR-011 proposés ; notion d'exposant ; lignes de facture ; correction du SQL ; légende, types et libellés sur tous les diagrammes	Conformité à la trame et à la checklist C4 ; notion d'exposant introduite par le client ; données fournies	décidée

Tableau 12 – Journal des évolutions du DAT (suite)

Version	Date	Évolution	Raison	Type
dat-v1.1	23/10/2026	Réponses de l'entrevue du 23/10 (Q20 à Q25) intégrées ; note datée ajoutée à l'ADR-003	Entrevue client	décidée
dat-v1.2	24/10/2026	VM du groupe (`162.38.111.31`) : Docker rootless sous un compte, accès réseau Polytech ou VPN ; HTTPS sans Let's Encrypt ; accès CI par clé dédiée (ADR-010 amendé)	Caractéristiques de la VM fournie	décidée
dat-v1.2	24/10/2026	ADR-012 : stockage des PDF (règles et factures)	Choix jusque-là implicite	décidée
dat-v1.2	24/10/2026	ADR-005 révisé et accepté : quatre couches (routes, contrôleurs, services, dépôts) au lieu de trois	Une seule responsabilité par couche ; organisation proposée par la trame	décidée
dat-v1.2	24/10/2026	ADR-013 : table `utilisateur_role` remplacée par `utilisateur.niveau` et `est_tresorier` ; modèle de données complété (liens vers `utilisateur`, attributs des jeux) ; annexe sur le rôle des tables	Simplification ; trésorier bloqué en lecture avec l'ancien modèle	décidée
dat-v1.2	24/10/2026	ADR-009, ADR-010 et ADR-011 acceptés par l'équipe	Relecture des ADR proposés	décidée
dat-v1.3	24/10/2026	Liste des endpoints corrigée : règles PDF, historique des contacts de suivi, droits du trésorier, forme publique des zones tarifaires, désactivation des comptes, suppressions refusées (409), routes de collection et d'élément séparées	Relecture de cohérence avec le DAT	correction

Tableau 12 – Journal des évolutions du DAT (suite)

Version	Date	Évolution	Raison	Type
dat-v1.3	24/10/2026	Visiteur sans compte confirmé par le client (entrevue du 23/10, Q26) ; explications des ADR-002 (liste blanche) et ADR-004 (numéro retiré) reformulées, sans changer les décisions	Précision du client ; clarté	décidée
dat-v1.3	24/10/2026	Codes HTTP par route dans la liste des endpoints ; route de changement de son propre mot de passe ; règle de copie d'un festival précisée ; tableau des niveaux C4 sécable	Relecture	décidée

Chaque version du journal correspond à un tag Git du même nom (`dat-v1`, `dat-v1.1`, `dat-v1.2`, `dat-v1.3`). Aucun écart subi à ce stade. Chaque écart sera décrit ainsi : écart constaté, cause, conséquence, ce qu'il faudrait faire pour revenir au plan.